Operatsion tizim va Linux asosiy amallari

Ubuntu Server 24.04 LTS

JDU Cyber Security / 2026 / ChatGPT tarjima qoralamasi

1-bob. Kompyuter va operatsion tizim

Ushbu bobdan keyin javob bera oladigan savollar

- Kompyuter qanday asosiy qismlar va kiritish-chiqarish qurilmalaridan iborat?
- Shaxsiy kompyuter va serverning o‘xshash hamda farqli tomonlari nimada?
- OS foydalanuvchi, ilovalar va apparat qismlari o‘rtasida qanday vazifani bajaradi?
- user space va kernel space qanday ajratilgan?
- Linux tizimi qanday ketma-ketlikda ishga tushadi va servicelarni qanday boshlaydi?
- user, host va process qanday aniqlanadi?

1.1 Kompyuter nimalardan iborat?

Kompyuter kiritilgan ma’lumotni program ko‘rsatmalariga binoan qayta ishlaydigan, natijani chiqaradigan yoki saqlaydigan qurilmadir. Noutbuk, stol kompyuteri, smartfon va bulutdagi virtual server tashqi ko‘rinishi hamda vazifasi bilan farqlanadi. Lekin ularning asosiy tuzilishi o‘xshash.

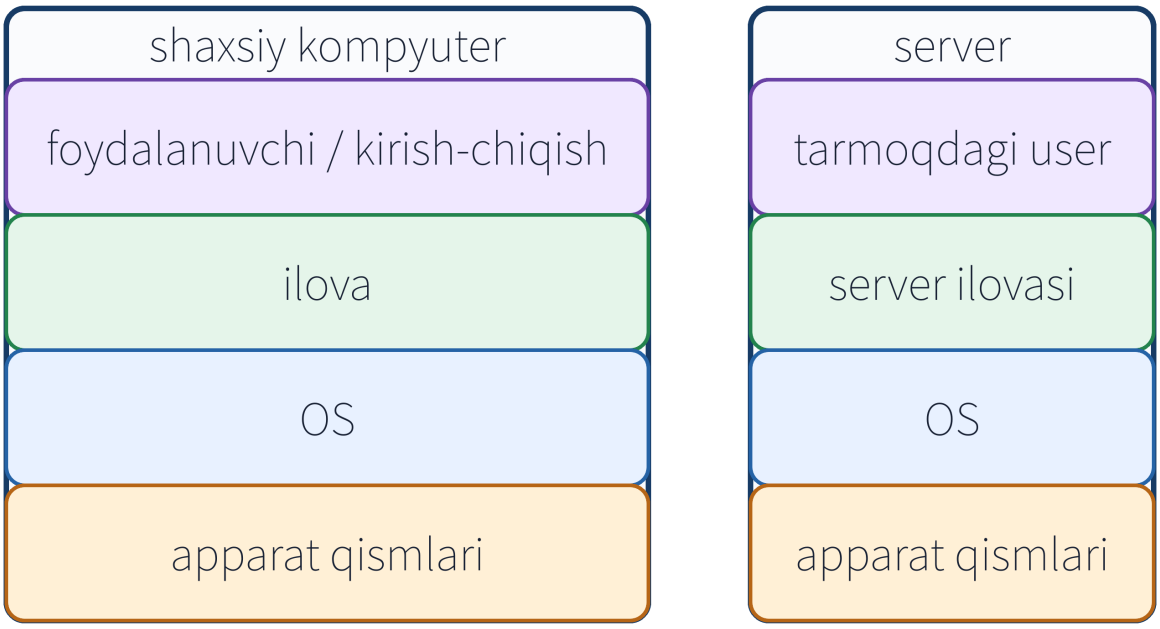
- CPU: program ko‘rsatmalarini o‘qiydi, hisob-kitob va qaror qabul qilish amallarini bajaradi.
- Tezkor xotira: ishlayotgan program va unga hozir kerak bo‘lgan ma’lumotlarni vaqtincha saqlaydi.
- Doimiy saqlash qurilmasi: elektr o‘chirilgandan keyin ham qolishi kerak bo‘lgan programlar, sozlamalar, hujjatlar, ma’lumotlar va loglarni saqlaydi. SSD va virtual disk bunga misol bo‘ladi.
- Kiritish qurilmasi: foydalanuvchi amali yoki tashqi ma’lumotni qabul qiladi. Klaviatura, sichqoncha va sensorli ekran shular jumlasidan.
- Chiqarish qurilmasi: natijani foydalanuvchiga ko‘rsatadi yoki eshittiradi. Monitor, karnay va printer bunga misol bo‘ladi.
- Tarmoq qurilmasi: boshqa kompyuterlar bilan ma’lumot almashadi.

Foydalanuvchi tugmani bosganda CPU yoki disk uning maqsadini o‘z-o‘zidan tushunmaydi. Kiritilgan amalni ilovaga yetkazish, ilovaga CPU va xotira ajratish, saqlangan filelarni o‘qish yoki yozish hamda natijani ekranga yoki tarmoqqa chiqarish uchun umumiy mexanizm kerak. Bu vazifaning markazida operatsion tizim (OS) turadi.

1.2 Shaxsiy kompyuter va server ham kompyuterdir

Shaxsiy kompyuter (PC) odatda uning oldida o‘tirgan odam tomonidan klaviatura, sichqoncha va monitor orqali boshqariladi. server esa tarmoq orqali boshqa kompyuterlarga funksiya yoki ma’lumot taqdim etadi. Veb-sahifani yetkazadigan veb-server, ma’lumotni saqlaydigan va qidiradigan ma’lumotlar bazasi serveri, nomlarni IP manzilga bog‘laydigan DNS serveri bunga misol bo‘ladi.

server shaxsiy kompyuterdan butunlay boshqa tamoyil asosida ishlamaydi. Ikkalasida ham CPU, xotira, saqlash qurilmasi, tarmoq, OS va ilovalar bor. Asosiy farq ularning vazifasi, unumdorligi, ulanish usuli va uzluksiz ishlash talabidadir. serverga doimiy klaviatura yoki monitor ulanmagan bo‘lishi mumkin. Bunday holda administrator uni boshqa kompyuterdan tarmoq orqali boshqaradi. AWS EC2 instancei ham AWS infratuzilmasida ishlaydigan virtual kompyuter. Ushbu kursda u server sifatida ishlatiladi.



1-1-rasm. Shaxsiy kompyuter va serverning vazifasi hamda boshqarish usuli farqlansa ham, ikkalasida ilovalar, OS va apparat qismlarining asosiy tuzilishi mavjud.

1.3 OS nimani boshqaradi?

Bir vaqtning o‘zida bir nechta ilova ishlaganda, har biri CPU yoki xotirani o‘zi uchun to‘liq egallashi yoki boshqa foydalanuvchining filelarini o‘qishi mumkin bo‘lmasligi kerak. OS apparat qismlari bilan ilovalar o‘rtasida turadi. U resurslardan foydalanishni muvofiqlashtiradi, umumiy imkoniyatlar yaratadi va himoyani ta’minlaydi.

Ushbu kitobda OSning vazifalari to‘rt guruhga bo‘linadi.

1. Ishlashni boshqarish: programni process sifatida ishga tushirish, CPUdan foydalanish navbati va process holatini boshqarish.
2. Xotira va filelarni boshqarish: processlarga xotira ajratish, saqlash qurilmasidagi ma’lumotni file va directory ko‘rinishida taqdim etish.
3. Foydalanuvchi va himoyani boshqarish: user, group va permission asosida kim nimani o‘qishi, o‘zgartirishi yoki ishga tushirishi mumkinligini aniqlash.
4. Aloqani boshqarish: processlar tarmoqdan foydalanishi uchun socket kabi umumiy imkoniyatlarni taqdim etish.

OS umumiy imkoniyatlarni bergani sababli ilova yaratuvchisi har bir kompyuter turidagi CPU va diskni boshqarish kodini boshidan yozishi shart emas.

1.4 GUI, CLI va command

Inson kompyuterga ko‘rsatma beradigan usul foydalanuvchi interfacei deb ataladi. Uning ikki keng tarqalgan turi bor.

- GUI (Graphical User Interface): oynalar, belgilar, menyular va tugmalarni ko‘rsatadi. Ko‘rsatmalar asosan sichqoncha yoki sensorli ekran orqali beriladi.
- CLI (Command Line Interface): ko‘rsatma matn sifatida kiritiladi, natija ham matn ko‘rinishida olinadi.

CLIga kiritiladigan har bir ko‘rsatma command deb ataladi. command nomidan keyin kerak bo‘lsa option va ishlov beriladigan obyekt yoziladi. Masalan, cat file ichidagi ma’lumotni ketma-ket o‘qib, standart chiqishga chiqaradigan commanddir.

```
cat /etc/os-release
```

Bu misolda cat — command nomi, /etc/os-release esa o‘qiladigan filening pathidir. Kiritilgan matnni shell talqin qiladi va cat programini ishga tushiradi. cat OSdan fileni o‘qishni so‘raydi. OS avval permissionni tekshiradi, keyin ma’lumotni beradi. command, terminal va shellning farqi hamda asosiy amallar 3-bobda batafsil ko‘rib chiqiladi.

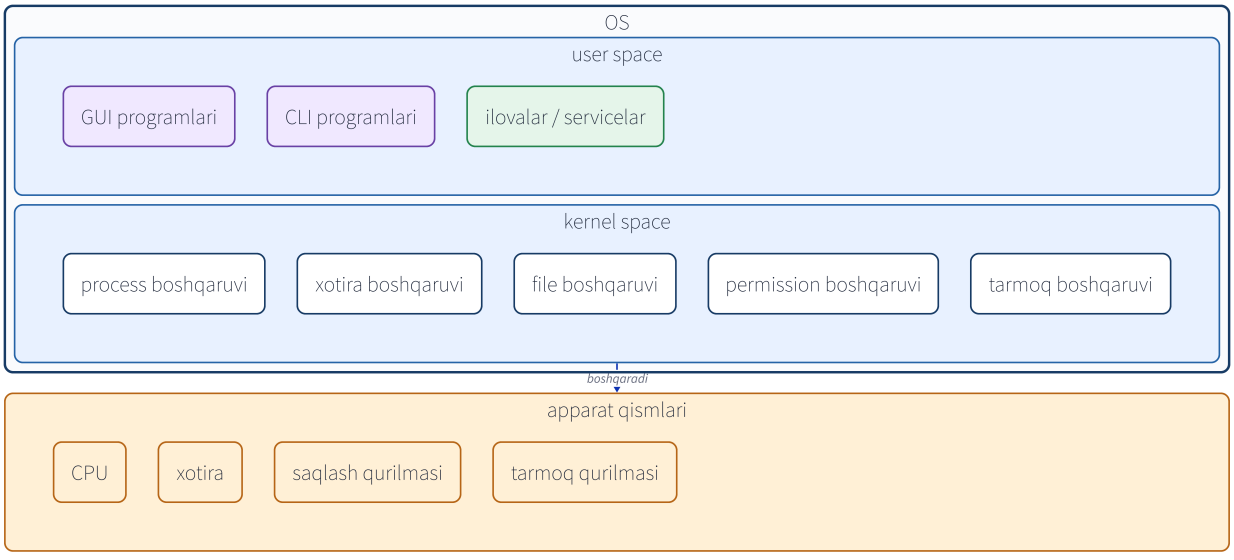
1.5 User space va kernel space

Linux tizimini tushunishda uni user space va kernel spacega ajratib tasavvur qilish foydali.

kernel — OSning markaziy qismi. U CPU, xotira, qurilmalar, file system, processlar, permissionlar va tarmoqni boshqaradi. kernel yuqori vakolat bilan ishlaydi. Shu sababli u oddiy ilovalardan ajratilgan kernel spaceda bajariladi.

GUIga tegishli programlar, terminal, shell, commandlar, veb-server va ma’lumotlar bazasi odatda user spaceda ishlaydi. GUI bilan CLI ikkita alohida “makon” emas. Ikkalasi ham user spacedagi boshqarish usullari va ularga tegishli programlardir. Ilovalar ham user spaceda ishlaydi va kerakli amallarni bajarishni kerneldan so‘raydi.

user spacedan kernel imkoniyatini chaqirish mexanizmi system call deyiladi. Ushbu kitobda system call raqamlari yoki ularni dasturlash tartibi o‘rganilmaydi. Uni “user spacedagi program himoyalangan kernel imkoniyatidan foydalanadigan kirish nuqtasi” deb tushunish yetarli.



1-2-rasm. Keng ma’nodagi OS muhiti user spaceda ishlaydigan programlar va apparat qismlarini boshqaradigan kernel spacedan iborat.

“OS” atamasi ba’zan faqat kernelga yaqin ma’noda, ba’zan esa asosiy commandlar va boshqaruv programlarini ham o‘z ichiga olgan butun tizim ma’nosida ishlatiladi. Farqlash zarur bo‘lsa, ushbu kitob “kernel” va “user spacedagi programlar” iboralarini aniq ishlatadi.

1.6 Linux va Ubuntu

Linux dastlab kernel nomidir. Amalda foydalaniladigan tizim uchun esa kernel bilan birga asosiy commandlar, shell, kutubxonalar, packagelarni boshqarish va servicelarni boshqarish vositalari ham kerak. Ularni o‘rnatish va yangilash mumkin bo‘lgan majmua shaklida jamlagan tizim Linux distributioni deb ataladi.

Ubuntu, Debian va Fedora — Linux distributionlariga misol. Amaliy mashg‘ulot muhiti bir xil bo‘lishi uchun ushbu kursda Ubuntu Server 24.04 LTS ishlatiladi. Biroq o‘qishning asosiy maqsadi Ubuntu nomini yodlash emas. Maqsad — Linux tizimlariga xos umumiy tushunchalar va amallarni o‘rganish. distributionlar tarixi va farqlari 2-bobda tushuntiriladi.

1.7 Kompyuter yoqilgandan service ishga tushguncha

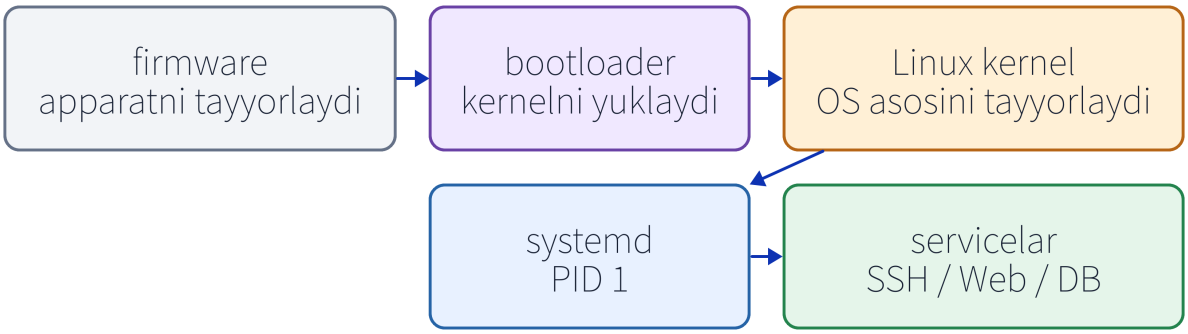
Saqlash qurilmasidagi program faqat saqlanib turgani uchun hech qanday ish bajarmaydi. program xotiraga yuklanib, CPU tomonidan bajarilayotgan holat process deyiladi. OS har bir processni farqlash uchun unga PID (process ID) raqamini beradi.

Linux tizimi taxminan quyidagi tartibda ishga tushadi.

1. Elektr yoqilgach, firmware CPU, xotira va saqlash qurilmasi kabi qismlarni tayyorlaydi.
2. Bootloader Linux kernelini xotiraga yuklaydi.
3. kernel xotira, qurilmalar va file systemdan foydalanish imkonini tayyorlaydi.
4. kernel user spacedagi birinchi boshqaruv processini ishga tushiradi.
5. Ubuntu Server 24.04da bu boshqaruv processi systemd bo‘lib, uning PID raqami 1 bo‘ladi.
6. systemd sozlamalar va o‘zaro bog‘liqliklarni o‘qiydi. Keyin tarmoq, SSH, loglarni boshqarish, veb-server va ma’lumotlar bazasi uchun kerakli servicelarni ishga tushiradi va kuzatadi.

service — foydalanuvchi har safar ekranda qo‘lda ishga tushirmasa ham, fon rejimida davomli imkoniyat berishi uchun boshqariladigan program. service ishga tushganda, haqiqiy ishni bajaradigan bir yoki bir nechta process ishlaydi. Masalan, ma’lumotlar bazasi serverida ma’lumotlar bazasi servicei boshqaruv obyekti bo‘ladi. Uning processlari xotirada ishlaydi, kerak bo‘lsa filelarni o‘qiydi yoki yozadi va tarmoqdan keladigan so‘rovni kutadi.

systemd ma’lumotlar bazasidagi so‘rovlarni yoki veb-sahifa tayyorlash ishini o‘zi bajarmaydi. U qaysi service qachon, qanday ketma-ketlikda va qaysi user nomidan ishga tushishini boshqaradi. PID, processlarning ota-bola munosabati va signallar 8-bobda, systemd, service va unitlar 10-bobda batafsil tushuntiriladi.



1-3-rasm. Kernel user space dagi birinchi processni ishga tushiradi, systemd esa kerakli service processlarini belgilangan tartibda boshlaydi.

1.8 Host va userni aniqlash

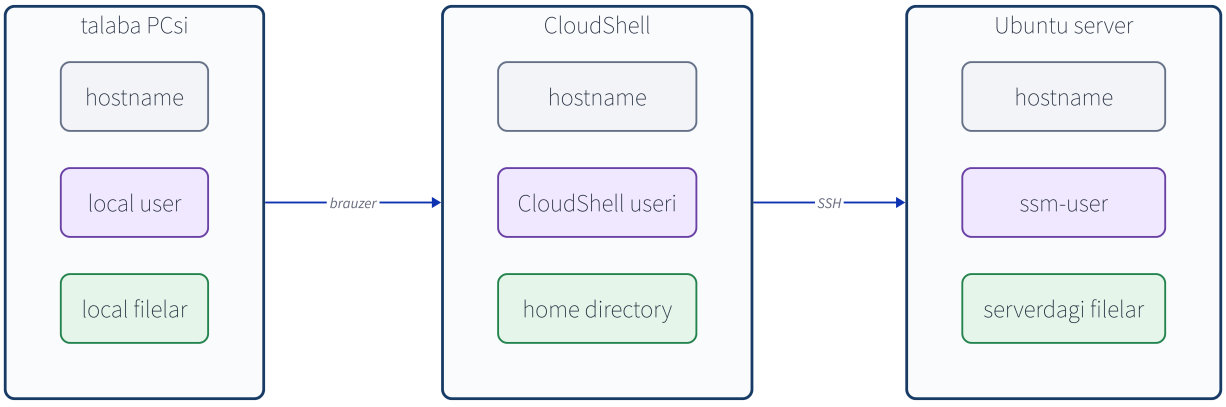
Tarmoqqa ulangan va mustaqil OS muhiti sifatida ishlaydigan kompyuter yoki virtual mashina host deb ataladi. Har bir hostni aniqlash uchun hostname beriladi. hostname — kompyuterning nomi, foydalanuvchining nomi emas.

Bitta hostda bir nechta user hisobi bo‘lishi mumkin. OS amalni bajarayotgan user va unga tegishli grouplarni raqamli IDlar orqali boshqaradi. userni aniqlovchi raqam UID (user ID), groupni aniqlovchi raqam GID (group ID) deyiladi. id commandi hozirgi user nomi, UID, primary group va supplementary grouplarni ko‘rsatadi.

Amaliy mashg‘ulotda talabaning kompyuteri, AWS CloudShell va Ubuntu EC2 kabi bir nechta host ishlatiladi. Ularning matn kiritish oynalari o‘xshash ko‘rinsa ham, har birining alohida OSi, userlari, processlari va filelari bor. Hozirgi user, hostname va ish directorysini tekshirib, aynan qaysi hostning qaysi joyida ishlayotganingizni bilib olasiz.

```
id
hostname
pwd
```

id user va group identifikatorlarini, hostname host nomini, pwd esa joriy ish directorysini ko‘rsatadi. userlarni boshqarish 6-bobda, file va pathlar 4-bobda, CloudShell bilan Ubuntu o‘rtasidagi aloqa 12-bobda batafsil ko‘riladi.



1-4-rasm. Har bir host alohida OS muhitiga ega; unda userlar, processlar va filelar mustaqil boshqariladi.

Bob yakunidagi savollar

1-savol. Shaxsiy kompyuter va serverning o‘xshash hamda farqli tomonlari nimada?

2-savol. GUI, CLI, user space va kernel space qanday bog‘langan?

3-savol. Program, process va service nimasi bilan farqlanadi?

4-savol. PID 1 ga ega systemd nima qiladi?

5-savol. Hostname bilan user nomi o‘rtasidagi farq nima?

Bob yakunidagi javoblar

1-savol

Javob: Ikkalasi ham CPU, xotira, saqlash qurilmasi, tarmoq, OS va ilovalarga ega kompyuterdir. Shaxsiy kompyuter odatda oldida o‘tirgan foydalanuvchining bevosita ishiga xizmat qiladi. serverning asosiy vazifasi esa tarmoq orqali boshqa kompyuterlarga imkoniyat yoki ma’lumot berishdir.

2-savol

Javob: GUI va CLI — foydalanuvchi ko‘rsatma beradigan ikki usul. Ularni amalga oshiradigan programlar odatda user spaceda ishlaydi. user spacedagi program system call orqali kernel space imkoniyatidan foydalanadi. kernel apparat qismlari va OS resurslarini boshqaradi.

3-savol

Javob: program — saqlash qurilmasidagi ko‘rsatmalar to‘plami. process — shu program xotiraga yuklanib, bajarilayotgan holat. service esa davomli imkoniyat berishi uchun systemd kabi vosita boshqaradigan birlik bo‘lib, uning haqiqiy ishini bir yoki bir nechta process bajaradi.

4-savol

Javob: Ubuntu Server 24.04da kernel ishga tushiradigan birinchi user space boshqaruv processi systemd bo‘lib, u PID 1 ga ega. systemd sozlamalar va bog‘liqliklarga ko‘ra servicelarni ishga tushiradi, to‘xtatadi va kuzatadi.

5-savol

Javob: hostname OS ishlayotgan kompyuter yoki virtual mashinani aniqlaydi. user nomi esa shu hostda ishlash huquqiga ega hisobni aniqlaydi. Bitta hostda bir nechta user bo‘lishi mumkin.

Foydalanilgan manbalar

- Linux Kernel Organization, The Linux Kernel documentation (2026-09-21 kuni tekshirilgan)
- Linux man-pages, intro(2) - introduction to system calls (2026-09-21 kuni tekshirilgan)
- Ubuntu Manpages, boot(7) - system bootup process (2026-09-21 kuni tekshirilgan)
- Ubuntu Server documentation, Ubuntu Server documentation (2026-09-21 kuni tekshirilgan)

2-bob. Unixdan hozirgi operatsion tizimlargacha

Nima uchun tarixni o‘rganamiz?

Maqsad sanalarni yodlash emas. Linux, macOS va Windowsda nima sababdan ayrim atamalar o‘xshash, ayrim commandlar esa boshqacha ekanini tushunish muhim. Shuningdek, “Linux Unixning asl kodini bevosita meros qilib olgan”, “macOS Linuxning bir turi”, “Windows ham Unixdan kelib chiqqan” kabi noto‘g‘ri tasavvurlardan qochish kerak.

2.1 Unix ilgari surgan g‘oyalar

Unix ustida ishlash 1969-yilda Bell Labsda boshlandi. Keyingi rivojlanish jarayonida u iyerarxik file system, process, bir nechta user bilan ishlash va kichik vositalarni birlashtirish kabi g‘oyalarni ommalashtirdi. Bu g‘oyalar hozirgi OSlarda ham uchraydi.



2-1-foto. Dastlabki Unix ishlab chiqilgan DEC PDP-7 rusumidagi saqlanib qolgan kompyuter. 2018-yilda AQShdagi muzeyda suratga olingan. Surat muallifi: ComputerGeek7066, Wikimedia Commons, CC BY-SA 4.0 (o‘zgartirilmagan).

Bu surat o‘sha davrda Bell Labsda ishlatilgan aynan shu qurilmaga tegishli emas. PDP-7 1960-yillarda ishlab chiqarilgan kompyuter turi bo‘lib, dastlabki Unix shu turdagi qurilmada ishlagan.

Ushbu kitobdagi amallar bilan bog‘liqlikni ko‘rish oson: filelarni directorylar iyerarxiyasida saqlash; programni process sifatida kuzatish; user va group orqali huquqlarni ajratish; grep natijasini boshqa command yoki filega uzatish. Bular Unixga o‘xshash tizimlarda uzoq vaqt shakllangan ishlash usullaridir.

“Unix” bitta command to‘plamining nomi emas. Tarixiy Unixdan ta’sirlangan barcha tizimlar bir xil manba kodiga yoki rasmiy UNIX savdo belgisi sertifikatiga ega degani ham emas. Odatda Unixga o‘xshash tuzilish va interfacega ega tizimlar Unix-like deb ataladi. Rasmiy sertifikatlash masalasini texnik g‘oyalarning o‘xshashligidan ajratib tushunish kerak.

2.2 GNU va erkin Unixga mos muhit

GNU Project 1983-yilda e’lon qilingan. Uning maqsadi Unix bilan mos, erkin foydalaniladigan dasturiy tizim yaratish edi. Loyiha doirasida shell, kompilyator va asosiy yordamchi programlar ishlab chiqildi. Hozirgi ko‘plab Linux distributionlarida Linux kerneli bilan birga GNU vositalari va boshqa ko‘plab programlar ishlaydi.

shell — foydalanuvchi kiritgan matnni o‘qiydigan, uni command sifatida talqin qiladigan va kerakli programni ishga tushiradigan user space programi. shell faqat bitta commandni boshlash bilan cheklanmaydi. U argumentlarni ajratadi, kirish va chiqishni qayta yo‘naltiradi, pipelarni ulaydi, muhit o‘zgaruvchilarini kengaytiradi va bir nechta commanddan iborat skriptni ketma-ket bajaradi. Shunday qilib, shell foydalanuvchi bilan ko‘plab commandlar o‘rtasidagi ishni tashkil qiladi.

Bash — GNU Project ishlab chiqadigan shelllardan biri. Nomi “Bourne-Again Shell” iborasidan kelib chiqqan. Bash Linux kerneli ham, terminal oynasi ham emas. Ubuntu terminaliga kiritilgan matnni Bash talqin qiladi va ls yoki cp kabi programlarni ishga tushiradi. cd kabi shellning o‘z holatini o‘zgartiradigan amal esa Bash ichiga o‘rnatilgan command sifatida bajariladi. terminal, shell va command farqi 3-bobda batafsil o‘rganiladi.

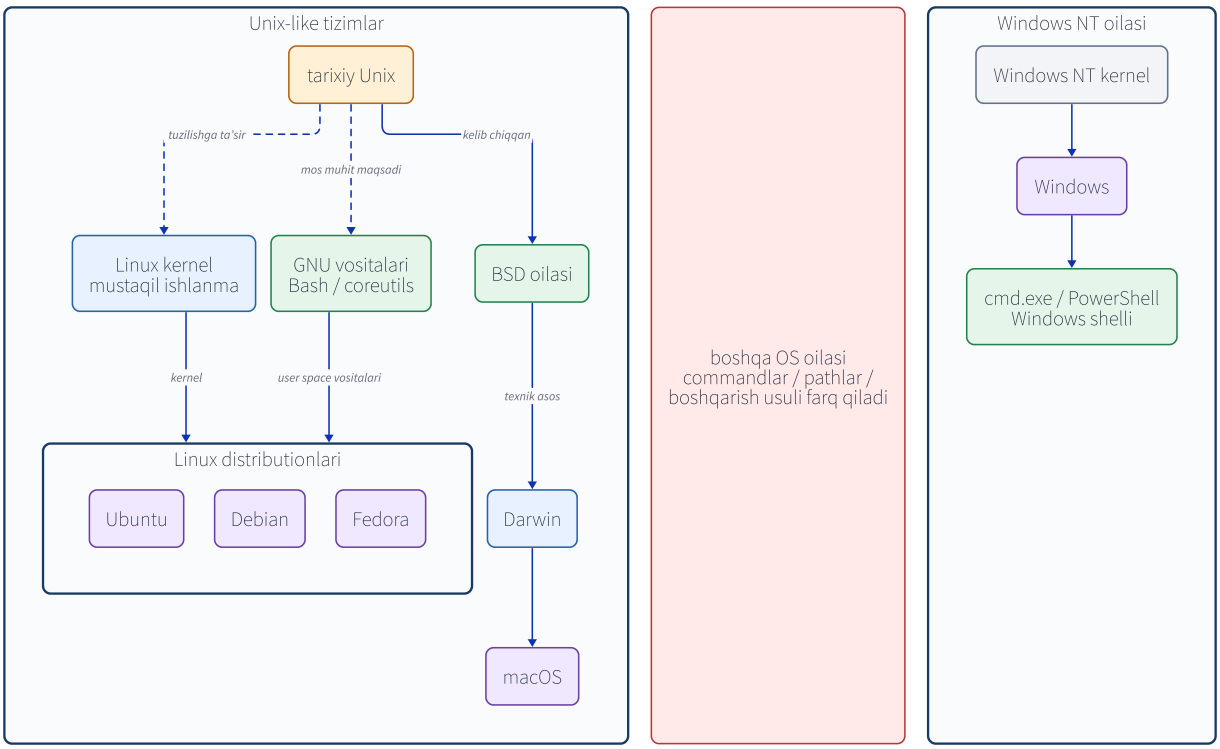
Bu yerda asosiy fikr: kernelning o‘zi talaba ishlata oladigan to‘liq tizim emas. Masalan, Bash shell vazifasini bajaradi, GNU coreutils esa ls va cp kabi asosiy commandlarni beradi. Bundan tashqari, package boshqaruvchisi, systemd, kutubxonalar, sozlama filelari va turli ilovalar birlashgandagina amalda foydalaniladigan tizim hosil bo‘ladi.

2.3 Linux — mustaqil yaratilgan kernel

Linux kernelini yaratish 1991-yilda boshlandi. Uning maqsadi Unixga o‘xshash imkoniyat va interfacelarni taqdim etish edi. Lekin Linux tarixiy Unixning manba kodidan bevosita ajralib chiqqan tarmoq emas. Shuning uchun OSlar kelib chiqishi chizmasida Unixdan Linuxga “manba kodi meros bo‘lgan” degan uzluksiz chiziq tortish noto‘g‘ri. G‘oyaviy ta’sir yoki moslik maqsadini uzilgan chiziq bilan ifodalash to‘g‘riroq.

Linux kerneli process, xotira, file system, qurilmalar va tarmoq kabi quyi layerlarni boshqaradi. Ubuntu esa Linux kernelini ko‘plab user space vositalari, packagelar, boshlang‘ich sozlamalar va chiqarilish siyosati bilan birlashtirgan distributiondir.

Windows NT oilasi esa Unixdan ajralib chiqqan OS emas. U Unixga o‘xshash tizimlardan boshqa yo‘nalishda ishlab chiqilgan. Windowsdagi cmd.exe va PowerShell ham shell, lekin ularning sintaksisi, ichki commandlari, path yozilishi va boshqarish usuli Bashdan farq qiladi. Nomi o‘xshash commandlar yoki bir nechta OSda ishlaydigan tashqi programlar mavjud. Shunga qaramay, Unixga o‘xshash tizimdagi har bir command Windowsda aynan shu tarzda ishlaydi deb hisoblamang. Bash Linuxning o‘zi emas; u Linux va macOS kabi tizimlarda foydalanish mumkin bo‘lgan shelllardan biridir.

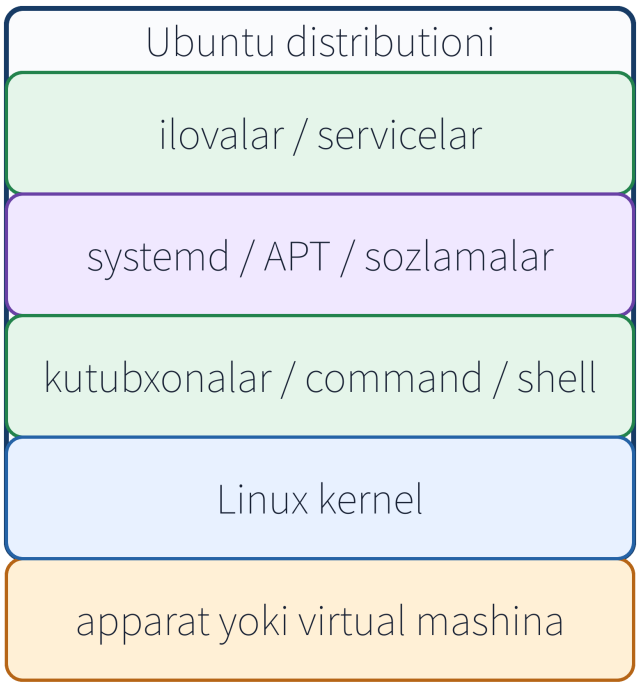


2-1-rasm. Unix-like va Unixga mos tizimlar Windows NT oilasidan alohida yo‘nalishdir; ularning commandlari bir-biriga to‘liq mos deb bo‘lmaydi. Linux Unixga o‘xshash interfacelarga ega mustaqil ishlanma, macOS esa Linuxdan kelib chiqmagan.

2.4 Distribution tanlaydi va birlashtiradi

Linux kernelidan foydalanadigan Ubuntu, Debian, Fedora va boshqa distributionlar bor. Har bir distribution qaysi dasturiy ta’minotni qo‘shish, package formati, yangilash siyosati, boshlang‘ich sozlamalar va qo‘llab-quvvatlash muddatini o‘zi belgilaydi.

Ushbu kursda amaliy muhit Ubuntu Server 24.04 LTS bilan bir xil qilinadi. Boshqa distributionlarda ham ls yoki grep ishlashi mumkin. Lekin packagelarni boshqarish commandlari, sozlama filelarining joyi, xavfsizlik funksiyalari va servicelarning boshlang‘ich sozlamalari farqlanishi mumkin. Red Hat/RHEL uchun yozilgan ko‘rsatmani tekshirmasdan Ubuntuga qo‘llamang.



2-2-rasm. Distribution Linux kernel, kutubxonalar, asosiy commandlar, tizim boshqaruvi, packagelar va ilovalarni birlashtiradi. Bu tuzilma Ubuntu va Linuxni to‘liq sinonim deb hisoblamaslikka yordam beradi.

2.5 macOSning asosi — Darwin

Hozirgi macOSning asosida Darwin turadi. Apple rasmiy hujjatlarida Darwin Mach, BSD va Apple yaratgan texnologiyalardan tuzilgani aytiladi. BSDdan kelgan process modeli, permission tizimi va tarmoq funksiyalari tufayli terminalda Unixga o‘xshash command hamda pathlardan foydalanish mumkin.

Ammo macOS Linux distributioni emas. Uning markazida Linux kerneli ishlamaydi. macOS shuningdek o‘z GUI tizimi va Applega xos dasturiy platformalarni ham o‘z ichiga oladi; ular Darwinning o‘zidan iborat emas. Qadimgi Classic Mac OS bilan Mac OS Xdan boshlangan Darwin asosidagi tizimni ham bir-biridan farqlash kerak.

2.6 Windowsni alohida yo‘nalish sifatida tushunish

Dastlabki oddiy foydalanuvchilarga mo‘ljallangan Windows turlarida MS-DOSga asoslangan yo‘nalish bo‘lgan (masalan, Windows 95/98). Hozirgi Windowsga bevosita olib kelgan Windows NT oilasi esa ulardan alohida, yangi tizim sifatida ishlab chiqilgan. Windows 2000, XP va bugungi Windowsni Unixdan kelib chiqqan oddiy shoxcha sifatida tasvirlash noto‘g‘ri.

Windowsda ham kernel mode va user mode, process, file permission, service, tarmoq socketi va CLI bor. Buning sababi — zamonaviy OSlar o‘xshash texnik muammolarni hal qilishi kerak. Bu Windows Unixning avlodi ekanini isbotlamaydi. PowerShelldan foydalanish yoki Windows Subsystem for Linux (WSL)ni ishga tushirish ham Windows kernelining o‘zi Linuxga aylanganini anglatmaydi.

2.7 O‘xshashlik va farqlarni qanday ko‘ramiz?

Mezoni	Ubuntu/Linux oilasi	macOS	Windows
Markaziy qism	Linux kerneli	Darwin (Mach, BSD va boshqalar)	Windows NT oilasidagi kernel
Odatdagi CLI	shell va Unixga o‘xshash vositalar	shell va BSD/Unix vositalari	PowerShell, cmd va boshqalar
path misoli	/home/user	/Users/user	C:\Users\user
Dastur o‘rnatish	distributionning package boshqaruvi	App Store, o‘rnatuvchi, package boshqaruv vositalari	Microsoft Store, o‘rnatuvchi, package boshqaruv vositalari
Ushbu kursdagi o‘rni	Amaliy mashg‘ulot muhiti	Taqqoslash uchun	Taqqoslash uchun

Bir xil maqsadga xizmat qiladigan funksiya ham turli OSlarda boshqa command nomi, boshlang‘ich sozlama yoki permission xatti-harakatiga ega bo‘lishi mumkin. Bu kursda Ubuntu orqali OSning asosiy tamoyillari o‘rganiladi. Keyin boshqa OSga o‘tganda “aynan shu commandni qayerdan topaman?” deb emas, “shu vazifani qaysi funksiya yoki mexanizm bajaradi?” deb izlash muhim.

Bob yakunidagi savollar

1-savol. “Kernel” va “distribution” atamalaridan foydalanib, Linux bilan Ubuntu farqini tushuntiring.

2-savol. Shell nima? Bash misolida tushuntiring.

3-savol. Nima uchun macOSni Linux distributioni deb bo‘lmaydi?

4-savol. Windowsda ham process va permission borligi uning Unixdan bevosita kelib chiqqanini isbotlaydimi? Sababini ayting.

Bob yakunidagi javoblar

1-savol

Javob: Linux — process, xotira va qurilmalarni boshqaradigan kernel. Ubuntu — Linux kernelini GNU vositalari, shell, kutubxonalar, package boshqaruvi va sozlamalar bilan birlashtirgan Linux distributioni.

2-savol

Javob: shell foydalanuvchi yozgan matnni command sifatida talqin qiladigan user space programidir. U programni ishga tushiradi, argumentlarni uzatadi, kirish-chiqishni qayta yo‘naltiradi, pipe va o‘zgaruvchilar bilan ishlaydi, skriptlarni bajaradi. Bash GNU Project ishlab chiqadigan shelllardan biri; u Linux kernelі emas.

3-savol

Javob: macOS Darwin asosida qurilgan. Uning markazida Mach va BSDdan kelgan texnologiyalar bor, Linux kernelі emas. Shuning uchun u Linux distributioni hisoblanmaydi.

4-savol

Javob: Yo‘q. process va permission zamonaviy OSlarga umumiy zarur funksiyalardir. Windows NT oilasi Unix-like tizimlardan alohida yo‘nalishda rivojlangan. cmd.exe va PowerShell sintaksisi, commandlari, pathlari va boshqarish usullari Bashga asoslangan Unix-like muhitdan farqlanadi.

Foydalanilgan manbalar

- Dennis M. Ritchie, The Evolution of the Unix Time-sharing System (2026-09-18 kuni tekshirilgan)
- Bell Labs, A history of computing at Bell Research Laboratories (1937-1975) (2026-09-24 kuni tekshirilgan; dastlabki Unix va PDP-7 munosabati)
- Wikimedia Commons, DEC PDP-7.jpg (2026-09-24 kuni tekshirilgan; ComputerGeek7066 surati, CC BY-SA 4.0; saqlangan kompyuter 2018-yilda suratga olingan)
- GNU Project, Initial Announcement (2026-09-18 kuni tekshirilgan)
- GNU Project, What is a shell? (2026-09-21 kuni tekshirilgan)
- GNU Project, What is Bash? (2026-09-21 kuni tekshirilgan)
- Linux Kernel Documentation, Introduction (2026-09-18 kuni tekshirilgan)
- Apple, Kernel Architecture Overview (2026-09-18 kuni tekshirilgan; eski arxiv hujjati faqat tarix va tuzilish uchun)
- Microsoft, Windows NT Build History (2026-09-18 kuni tekshirilgan; tarixiy manba)
- Microsoft Learn, Windows commands (2026-09-21 kuni tekshirilgan)

3-bob. GUI, CLI, terminal va shell

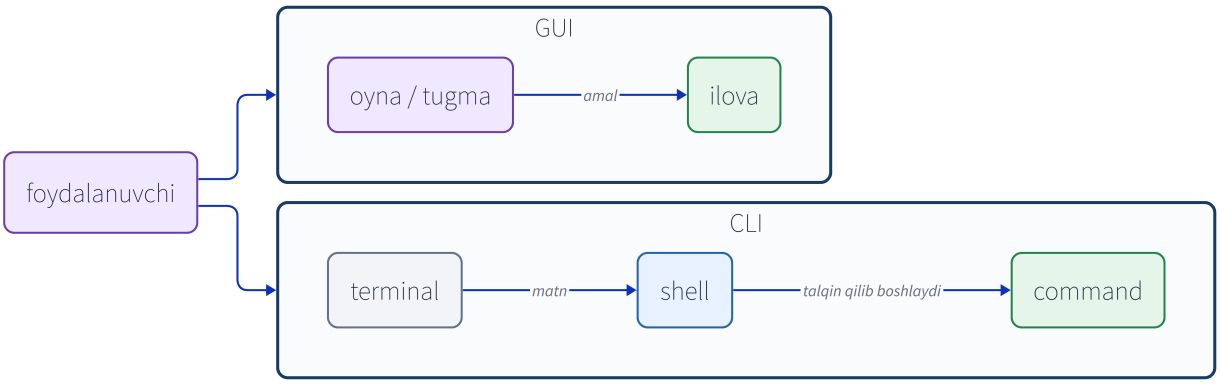
3.1 Boshqarish oynasi va amalni bajaruvchi programni ajratish

GUI (Graphical User Interface) — oyna, belgi, menyu va ko‘rsatkich yordamida ko‘rib boshqarish usuli. CLI (Command Line Interface) — commandlarni matn sifatida kiritish va natijani matn ko‘rinishida olish usuli. Ikkalasi ham odamning kompyuterga ko‘rsatma berishi uchun kirish nuqtasidir. GUI ishlashi uchun u ichkarida albatta CLIga aylanishi kerak degan qoida yo‘q.

terminal — matn kiritish va natijani ko‘rsatish oynasi yoki ulanish muhiti. shell esa kiritilgan matnni o‘qiydigan, commandni talqin qilib bajaradigan program. Ushbu amaliy muhitda asosan Bash shelli ishlatiladi.

```
Odam (operator) → terminal → Bash (shell) → command processi → kernel imkoniyati
```

terminal va shell bir xil narsa emas. terminal matn almashiladigan joy; shell esa kiritilgan matnning ma’nosini aniqlab, amalni bajaradigan program.



3-1-rasm. GUI va CLI parallel kirish usullaridir. CLI ishlatilganda terminal va shell vazifalarini farqlash kerak.

3.2 Prompt matn kiritiladigan joyni ko‘rsatadi

shell userdan yangi ko‘rsatma kutayotganda ko‘rsatadigan matn prompt deyiladi. Unda ko‘pincha user nomi, hostname va joriy ish directorysi ko‘rsatiladi. Aniq ko‘rinishi sozlamaga bog‘liq.

Darslikdagi quyidagi $ — prompt namunasi. Uni foydalanuvchi o‘zi kiritmaydi.

```
$ pwd
/home/ssm-user
```

Kiritiladigan command — pwd. Keyingi qator uning chiqishiga misol. Kitobdagi ayrim misollarda kiritilgan matnni natijadan ajratish uchun $ ko‘rsatiladi.

3.3 Command, argument va option

Asosiy tuzilishni quyidagicha o‘qing.

```
command  option  argument
ls       -l      /srv
```

ls — bajariladigan command; -l — batafsil ko‘rsatishni tanlaydigan option; /srv — ishlov beriladigan obyektni bildiradigan argument. Ko‘p commandlarda qisqa optionlar birlashtiriladi, masalan ls -la. Lekin bir xil harf har bir commandda bir xil vazifani anglatmaydi. Masalan, grep -n satr raqamlarini ko‘rsatadi, tail -n 5 esa ko‘rsatiladigan satrlar sonini belgilaydi.

command satrida bo‘sh joy elementlarni ajratadi. Shu sababli ichida bo‘sh joy bor matnni bitta argument sifatida uzatish uchun uni qo‘shtirnoq yoki apostrof bilan o‘rash kerak.

```
printf '%s\n' 'JDU SSH transfer test'
```

Qo‘shtirnoqlar, o‘zgaruvchilarni kengaytirish va chiqishni qayta yo‘naltirish 5-bobda batafsil o‘rganiladi.

3.4 Shell builtin va tashqi command

shellga kiritiladigan commandlar ikki asosiy turga bo‘linadi: shell builtin va tashqi command.

shell builtin — Bashning o‘ziga kiritilgan funksiya. Uni bajarish uchun alohida ishlatiladigan fileni topib ishga tushirish shart emas. Masalan, cd hozir ishlayotgan Bashning o‘z ish directorysini o‘zgartiradigan shell builtindir.

Tashqi command esa saqlash qurilmasidagi programdir; shell uni topib ishga tushiradi. ls, mkdir va grep Ubuntu o‘rnatilganda ko‘pincha mavjud bo‘ladi. Lekin ular Bashning ichki funksiyasi emas, tashqi programdir. Demak, boshidan mavjud bo‘lish va shell ichiga kiritilgan bo‘lish — ikki boshqa tasnif.

Ba’zi tashqi commandlar OS bilan birga keladi, boshqalari kerak bo‘lganda qo‘shimcha o‘rnatiladi. Masalan, python3 Python programlarini ishga tushiradigan tashqi command. U faqat Python o‘rnatilgan muhitda ishlaydi. Ubuntu turiga qarab oldindan mavjud bo‘lishi yoki keyin package sifatida o‘rnatilishi mumkin. package o‘rnatish 9-bobda ko‘riladi.

```
type cd
type mkdir
type python3
```

type kiritilgan nomni Bash qanday talqin qilishini ko‘rsatadi. shell builtin bo‘lsa shu haqida xabar beradi; tashqi command bo‘lsa uning bajariladigan filei joylashgan pathni ko‘rsatadi. Nom topilmasa, hozirgi muhitda bu commanddan foydalanib bo‘lmaydi.

type natijasida alias ham chiqishi mumkin. alias — uzun yoki tez-tez ishlatiladigan commandga qisqa boshqa nom berish imkoniyati. Masalan, ayrim muhitlarda ll nomi ls -alFga bog‘langan. Bunday nomning mavjudligi va mazmuni muhitga qarab o‘zgaradi.

Tashqi command kiritilganda, shell PATH sozlamasida sanab o‘tilgan directorylarni ketma-ket ko‘rib, mos nomli bajariladigan fileni izlaydi. Uning joylashgan manzilini oldindan yodlash kerak emas. Avval type orqali nom shell builtin, tashqi command yoki alias ekanini tekshiring.

3.5 Shell commandni qanday bajaradi?

shell kiritilgan matnni o‘zgartirmasdan kernelga yubormaydi. Taxminan quyidagi amallarni bajaradi.

1. Matnni command nomi, option va argumentlarga ajratib talqin qiladi.
2. Qo‘shtirnoq qoidalari, muhit o‘zgaruvchilari, $(...) orqali command natijasini qo‘yish va wildcardlarni tegishli qoida bo‘yicha kengaytiradi.
3. shell builtin bo‘lsa o‘zida bajaradi; tashqi command bo‘lsa bajariladigan fileni qidiradi.
4. Kerakli kirish-chiqish oqimlarini qayta yo‘naltirish va pipe orqali ulashga tayyorlaydi.
5. processni boshlaydi; u oldingi planda tugashini kutadi yoki orqa planda ishlashda davom etadi.
6. processning exit status qiymatini oladi.

Bu tartib xato qayerda yuz berganini aniqlashga yordam beradi. command topilmasa, uning nomi va tashqi program qayerdan qidirilganini tekshiring. Kutilgan obyektga amal qilinmasa, option va argumentlar to‘g‘ri talqin qilinganini tekshiring. Kirish-chiqish 5-bobda, processlar 8-bobda batafsil ko‘riladi.

3.6 Qiyin vaziyatda foydali amallar va tekshirish commandlari

CLI ishlatayotganda qayerda turganingizni yoki commandning vazifasini bilmay qolsangiz, taxmin bilan davom etish o‘rniga holatni tekshiring.

Amal yoki command	Nimani bilib olasiz?
Ctrl+C	Hozirgi amalga to‘xtash signali yuboradi va promptga qaytishga yordam beradi.
pwd	Joriy ish directorysini ko‘rsatadi.
cd directory_name	Ish directorysini o‘zgartiradi.
ls	Joriy directorydagi file va directorylarni ko‘rsatadi.
grep 'search_text' file_name	filedan ko‘rsatilgan matn bor satrlarni izlaydi.
type command_name	Nom shell builtin, tashqi command yoki alias ekanini aniqlaydi.
help command_name	Bash shell builtini uchun tushuntirishni ko‘rsatadi.
exit	Hozirgi shelldan chiqib, uni boshlashdan oldingi holatga qaytadi.

pwd, cd va ls “qayerda, nimani boshqaryapman?” savoliga javob topishda asosiy vositalardir. grep katta matn yoki logdan kerakli satrni qidiradi. type va help commandning turi va ishlatilishini aniqlashga yordam beradi. grep va ls foydali tekshirish vositalari bo‘lsa ham, shell builtin emas; yuqoridagi jadvaldagi hamma amal shell ichiga kiritilgan degani emas.

3.7 Manual va helpdan foydalanish

Tashqi command uchun man, Bash shell builtini uchun esa help ma’lumot olishning boshlang‘ich nuqtasidir.

```
man ls
help cd
grep --help
```

Manualdagi NAME bo‘limi commandning qisqa maqsadini, SYNOPSIS yozilish tartibini, DESCRIPTION ishlashini, OPTIONS esa optionlarning ma’nosini ko‘rsatadi. Sintaksisdagi [ ] odatda “ixtiyoriy” degan belgi; bu qavslarning o‘zini kiritish kerak emas. man oynasidan chiqish uchun q tugmasini bosing.

3.8 Exit status orqali natijani aniqlash

command ekranga chiqargan matndan tashqari, natijani bildiradigan kichik butun sonni shellga qaytaradi. Bu son exit status yoki exit code deyiladi. Unix-like tizimlarda 0 odatda muvaffaqiyatli yakunlanishni, 0dan boshqa son esa muvaffaqiyatsizlik yoki xatoni bildiradi.

```
test -e /etc/os-release
echo $?
```

$? oxirgi bajarilgan commandning exit statusini saqlaydigan maxsus o‘zgaruvchi. Keyingi command bajarilishi bilan bu qiymat yangilanadi. Shuning uchun kerakli commanddan darhol keyin tekshirish zarur. 0dan boshqa qiymatlarning hammasi bir xil sababni anglatmaydi. Masalan, grep mos satr topilmaganda 1, file yo‘qligi kabi bajarish xatosida 2 qaytaradi. Faqat ekrandagi xabarga emas, natija qanday baholanishiga ham qarang.

Bob yakunidagi savollar

1-savol. Terminal va shell vazifalari nimasi bilan farqlanadi?

2-savol. Nima uchun cd shell builtin bo‘lishi kerak?

3-savol. Nima uchun “boshidan mavjud command” va “shell builtin” bir xil tushuncha emas?

4-savol. Command turini qanday tekshirasiz?

Bob yakunidagi javoblar

1-savol

Javob: terminal matn kiritiladigan va natija ko‘rsatiladigan oyna yoki ulanish muhiti. shell esa kiritilgan matnni talqin qilib, ichki funksiyani bajaradigan yoki tashqi programni ishga tushiradigan program.

2-savol

Javob: cd aynan hozir ishlayotgan shellning ish directorysini o‘zgartirishi kerak. Agar faqat alohida tashqi process o‘z joyini o‘zgartirsa, asl shellning joriy joyi o‘zgarmaydi.

3-savol

Javob: Boshidan mavjud tashqi programlar ham bor. Masalan, mkdir Ubuntuda ko‘pincha oldindan mavjud, lekin Bash ichiga kiritilgan emas; u tashqi command. cd esa Bashning shell builtinidir.

4-savol

Javob: type command_name yordamida nom shell builtin, tashqi command yoki alias ekanini bilib olish mumkin.

Foydalanilgan manbalar

- Ubuntu Server documentation, Welcome to the terminal (2026-09-18 kuni ko‘rilgan; sahifaning hozirgi tuzilishi nashrdan oldin qayta tekshiriladi)
- GNU Project, Bash Reference Manual (2026-09-18 kuni matnni yuklashda timeout bo‘lgan; yozilgan ma’lumot Ubuntu 24.04dagi help va man bash orqali ham tekshiriladi)

4-bob. File, directory, path va tahrirlash

4.1 File ichidagi ma’lumot va metadata

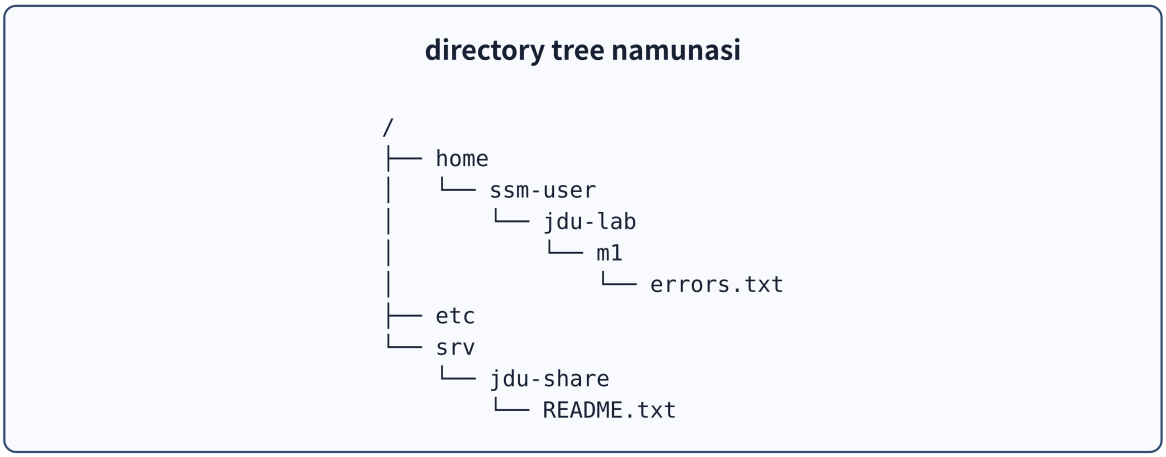
file faqat ichidagi matn yoki ma’lumotdan iborat emas. Uning nomi, turi, egasi (owner), groupi, permissioni, hajmi va oxirgi o‘zgartirilgan vaqti kabi metadatasi ham bor. cat bilan ko‘rsatilganda mazmuni aynan bir xil bo‘lgan ikkita filening egasi yoki permissioni farq qilsa, OS ularni turli holatdagi obyektlar deb hisoblaydi.

```
ls -l /srv/jdu-share/README.txt
stat /srv/jdu-share/README.txt
```

ls -l directorydagi asosiy ma’lumotlarni ro‘yxat shaklida ko‘rsatadi. stat esa bitta belgilangan obyektning metadatasini batafsil ko‘rsatadi.

4.2 Directory nomni obyektga bog‘laydi

directoryni shunchaki “ichiga filelar solingan quti” deb tushunish to‘liq emas. Aslida u file va boshqa directorylarning nomini tegishli obyektlarga bog‘lab, iyerarxiya hosil qiladigan maxsus turdagi filedir. /srv/jdu-share/README.txt pathi eng yuqoridagi / directorysidan boshlanib, srv, jdu-share, so‘ng README.txt nomlarini ketma-ket bosib o‘tadigan manzildir.



4-1-rasm. Tree commandi chiqishiga o‘xshash iyerarxik chizma. Shoxlarni ketma-ket kuzatib, file joyini aniqlash mumkin.

Linuxda butun tizim yuqorisida / joylashgan yagona directory tree ko‘rinishida taqdim etiladi. Turli fizik disklar yoki virtual file systemlar ham ma’lum directoryga ulanib, butun tizimning bir qismidek ko‘rsatilishi mumkin. Bunday ulash mount deyiladi. Bu bobda faqat g‘oya tushuntiriladi. Diskni formatlash va mountni amalda bajarish keyingi saqlash mavzusida o‘rganiladi.

root directory — butun directory treening boshlanish nuqtasi, ya’ni /. home directory — har bir user o‘z filelarini saqlaydigan joy. Masalan, ssm-userning home directorysi /home/ssm-user. Unga /dan boshlab home, keyin

ssm-user nomi orqali boriladi. /home bir nechta userning home directorylarini o‘z ichiga oladigan yuqori directory; u ssm-userning shaxsiy home directorysi emas. / va /home/ssm-userning vazifasi ham, joyi ham boshqa. Nomi o‘xshash /root ham / root directorysi bilan bir xil joy emas.

4.3 Absolute path va relative path

/ belgisi bilan boshlanadigan path absolute path deyiladi. Hozir qaysi directoryda bo‘lishingizdan qat’i nazar, u bir xil joyni ko‘rsatadi.

```
/srv/jdu-share/README.txt
```

/ bilan boshlanmaydigan path relative path deyiladi. U joyni hozirgi ish directorysiga nisbatan belgilaydi.

```
cd /srv/jdu-share
cat README.txt
```

pwd hozirgi ish directorysini ko‘rsatadi. . — “hozirgi directory”, .. — “bir pog‘ona yuqoridagi directory”. ~ esa shell tomonidan hozirgi userning home directorysiga kengaytiriladi.

```
cd ~/jdu-lab/m1
pwd
cd ..
pwd
```

~ root directory /ni emas, aynan hozirgi userning home directorysini anglatadi. Shu sababli ~ bor pathning aniq joyi user va hostga qarab o‘zgaradi. Masalan, ssm-user uchun ~ /home/ssm-user bo‘lsa, su - jduops bilan user almashtirilgach, u jduopsning home directorysini (masalan, /home/jduops) ko‘rsatadi. Boshqa hostga ulanganda ham o‘sha hostdagi hozirgi userning home directorysi ko‘rsatiladi. Hozirgi joyni pwd, home directoryni esa printf '%s\n' "$HOME" bilan tekshirish mumkin. HOME shu manzilni saqlaydigan muhit o‘zgaruvchisidir; o‘zgaruvchilar 5-bobda tushuntiriladi.

4.4 Ubuntuda ko‘p uchraydigan directorylar

path	Ushbu kitobdagi vazifasi
/home	Odatdagi userlarning home directorylari joylashadi.
/etc	Tizim va servicelarning sozlama hamda identifikatsiya filelari joylashadi. /etc/os-release ham shu yerda.
/usr/bin	Ko‘p oddiy commandlarning bajariladigan filelari saqlanadi.
/var/log	Tizim va servicelarning log filelari joylashadi. Barcha loglar aynan shu yerda bo‘lishi shart emas.
/srv	Tizim yoki service taqdim etadigan ma’lumotlar uchun joy. Amaliy mashg‘ulotda umumiy directory va veb-kontent shu yerda.
/tmp	Vaqtinchalik filelar uchun directory. Ko‘pchilik undan foydalana oladi, lekin u doimiy saqlash joyi emas.
/proc	kernel process va tizim holatini file ko‘rinishida ko‘rsatadigan virtual file system.

Bu jadval “barcha filelar faqat shu joyda bo‘lishi shart” degan qat’iy qoida emas. Tafsilotlar distribution siyosati va ilova tuzilishiga qarab farqlanishi mumkin.

4.5 Yaratish, nusxalash, ko‘chirish va o‘chirish

```
mkdir -p ~/jdu-lab/p1/practice01/config
cp inbox/config/training.conf practice01/config/training.conf
mv old-name.txt new-name.txt
rm unwanted.tmp
```

- mkdir -p pathda yetishmayotgan yuqori directorylarni ham yaratadi. Belgilangan directory oldindan bor bo‘lsa ham xato bermaydi.
- cp SOURCE DESTINATION file yoki directorydan nusxa oladi. Manba (SOURCE) va manzil (DESTINATION) tartibini adashtirmang.
- mv fileni ko‘chirish yoki nomini o‘zgartirish uchun ishlatiladi.
- rm fileni o‘chiradi. Oddiy GUI dagi “savatcha” orqali o‘tmaydi; odatda darhol o‘chadi. Ishlatishdan oldin pwd va ls bilan manzilni tekshiring.

Boshlang‘ich amaliyotda ogohlantirishsiz butun directoryni o‘chirishi mumkin bo‘lgan xavfli rm -rfdan foydalanilmaydi. Masalan, ortiqcha .tmp filelarni o‘chirishdan oldin find bilan ro‘yxatni ko‘ring.

```
find ~/jdu-lab/p1/practice01 -type f -name '*.tmp' -print
```

Yakka qo‘shtirnoq ichidagi '*.tmp' shellning wildcardni oldindan kengaytirishiga yo‘l qo‘ymaydi. Natijada qidiruv namunasi findga o‘z holicha yetadi. Topilgan filelarni ko‘zdan kechiring, keyin ularni bittadan yoki aniq ko‘rsatilgan xavfsiz doirada o‘chiring.

4.6 Boshqa tekshirish commandlari

```
ls -la
ls -ld /srv/jdu-share
tree ~/jdu-lab/m1/case01
stat -c '%U:%G %a %n' /srv/jdu-share/README.txt
```

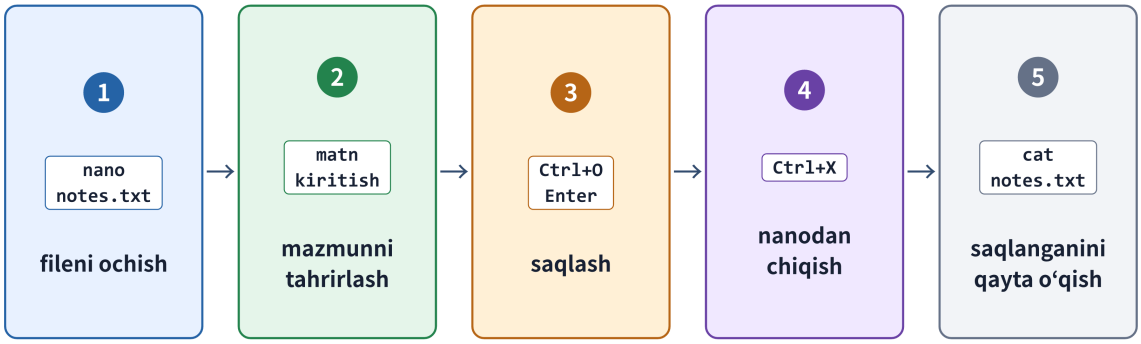
- ls -la hozirgi directorydagi barcha obyektlarni, jumladan nomi . bilan boshlanadigan yashirin filelarni batafsil ro‘yxatda ko‘rsatadi.
- ls -ld DIR ichidagi filelarni emas, ko‘rsatilgan directoryning o‘z metadatasini ko‘rsatadi.
- tree directory iyerarxiyasini daraxt ko‘rinishida ko‘rsatadi. U amaliy muhitni tayyorlash paytida oldindan o‘rnatiladi.
- stat -c kerakli maydonlarni belgilangan formatda chiqaradi. %U — egasi bo‘lgan user nomi, %G — egasi bo‘lgan group nomi, %a — sakkizlik sanoq tizimidagi permission mode, %n — file nomi yoki path.

Hammasini bitta command orqali bilishga urinmang. Iyerarxiya kerakmi (tree), ichidagi obyektlar ro‘yxatimi (ls), yoki atribut va metadata tafsilotlarimi (stat) — maqsadga mos vositani tanlang.

4.7 Nano bilan tahrirlash

nano FILE terminal ichida matnli fileni tahrirlash uchun sodda editordir. Ushbu kitobda tahrirlashdan oldin kerakli directoryga o‘ting, pwd va ls orqali joy hamda fileni tekshiring.

```
cd ~/jdu-lab/m0
pwd
nano observation.env
```



4-2-rasm. Nanonning asosiy amallari. Oynaning pastki qismidagi ^ belgisi Ctrl tugmasini bildiradi.

1. Yo‘nalish tugmalari bilan kursorni kerakli joyga olib boring va oddiy klaviatura bilan matnni tahrirlang.
2. Ctrl+Oni bosib, fileni yozish (saqlashga tayyorlash) amalini boshlang.
3. Eng pastki satrda file nomi chiqqanda, path to‘g‘riligini tekshirib Enterni bosing.
4. Ctrl+Xni bosib, nano editoridan chiqing.
5. cat observation.env kabi command bilan natijani qayta o‘qing va saqlanganini tekshiring.

Saqlanmagan o‘zgarish bo‘lsa, chiqishda saqlash yoki rad etish haqida savol beriladi. Inglizcha oynada Y (Yes) saqlash, N (No) saqlamasdan chiqish, Ctrl+C esa so‘rovni bekor qilishni bildiradi. Ekrandagi xabarni o‘qimasdan tugmalarni ketma-ket bosmang.

4.8 Yuqori directory permissioni ham muhim

file o‘qilayotganda OS pathni boshidan oxirigacha kuzatadi. Yo‘ldagi har bir directory uchun hozirgi userning undan o‘tish permissioni borligini tekshiradi. Birgina directorydan o‘tish mumkin bo‘lmasa, filening o‘zida o‘qish permissioni r bo‘lsa ham uni o‘qib bo‘lmaydi. namei -l pathdagi har bir yuqori directory va oxirgi fileni alohida ko‘rsatadi; ularning egasi va permissionini tekshirish mumkin.

```
namei -l /srv/jdu-web/index.txt
```

directory uchun bajarish permissioni x program fileni ishga tushirishdan boshqa ma’noga ega. Bu directorydan o‘tib, uning ichidagi nomga murojaat qilish huquqidir. Tafsilotlar 7-bobda tushuntiriladi.

Bob yakunidagi savollar

1-savol. Root directory va home directory o‘rtasidagi farq nima?

2-savol. `ls -l DIR` va `ls -ld DIR` nimalarni ko‘rsatadi?

3-savol. Filening o‘zida o‘qish permissioni `r` bo‘lsa ham o‘qib bo‘lmasa, `namei -l`da nimani tekshirasiz?

Bob yakunidagi javoblar

1-savol

Javob: root directory / butun tizimdagi directory treening boshlanishi. home directory har bir userning ish joyi; masalan, ssm-user uchun /home/ssm-user. ~ hozirgi userning home directorysini bildiradi.

2-savol

Javob: ls -l DIR DIR ichidagi obyektlarni batafsil ko‘rsatadi. ls -ld DIR esa DIRning o‘zi haqidagi ma’lumotni chiqaradi.

3-savol

Javob: path boshidan oxirgi filegacha bo‘lgan har bir yuqori directoryda hozirgi user uchun o‘tish permissioni x borligini tekshiring. Oraliqdagi bittasidan ham o‘tib bo‘lmasa, oxirgi fileda r bo‘lishi yetarli emas.

Foydalanilgan manbalar

- Linux man-pages, path_resolution(7) (2026-09-18 kuni tekshirilgan)
- Ubuntu 24.04 amaliy muhitidagi man hier, man find, man stat va nano oynasi (nashrdan oldin amalda tekshiriladi)

5-bob. Matn va kirish-chiqishni birlashtirish

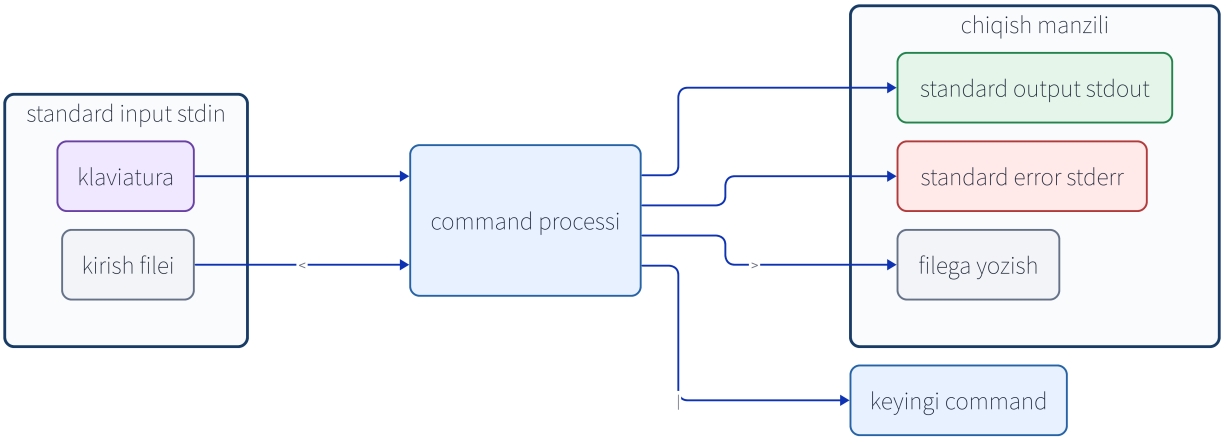
5.1 Nima uchun sozlama va loglar ko‘pincha matn bo‘ladi?

Ubuntu va boshqa Linux tizimlarida ko‘plab sozlamalar hamda loglar oddiy matn sifatida saqlanadi. Shunda maxsus GUI bo‘lmasa ham ularni o‘qish, solishtirish va qidirish oson. Natijani boshqa commandga ham uzatish mumkin. Lekin har bir sozlama yoki log oddiy matnli file bo‘lishi shart emas. Masalan, systemd journal ma’lumotni ikkilik formatda saqlaydi va uni maxsus command orqali ko‘rish mumkin.

5.2 Standard input, standard output va standard error

Har bir process uchun ma’lumot almashishning uchta asosiy oqimi bor: stdin (standard input), stdout (standard output) va stderr (standard error).

- stdin: odatdagi kirish ma’lumoti. Klaviatura yoki avvalgi command chiqishidan ma’lumot qabul qiladi (file descriptor 0).
- stdout: odatdagi natija. terminalda ko‘rsatilishi, keyingi commandga yoki filega uzatilishi mumkin (file descriptor 1).
- stderr: xato xabari yoki ogohlantirish. Odatdagi natija (stdout)dan alohida oqimdir (file descriptor 2).



5-1-rasm. Programning uchta standart oqimi. > odatda stdoutni filega, | esa chapdagi commandning stdoutini o‘ngdagi commandning stdiniga ulaydi.

cat incident.log bajarilganda file ma’lumot manbasi, terminal esa stdoutni ko‘rsatadigan joy bo‘ladi. filening o‘zi terminalga “uchib o‘tmaydi”. cat processi file mazmunini o‘qib, matnni stdoutga yozadi.

5.3 Grep bilan kerakli satrlarni tanlash

```
grep 'ERROR' incident.log
```

grep kirishni satrma-satr tekshiradi va berilgan namunaga mos kelgan satrlarni to‘liq holda stdoutga chiqaradi.

```
grep -F 'ERROR' incident.log
```

`-F` qidiruv namunasini `regular expression` emas, oddiy qat'iy matn deb qabul qiladi. `ERROR` kabi oddiy so‘zda natija ko‘pincha bir xil. Lekin maxsus belgilarni qo‘shimcha qochirishsiz aynan o‘zicha qidirmoqchi bo‘lsangiz, `-F` foydali.

`grep -n` mos satr oldiga satr raqamini ham qo‘shadi. Faqat asl satrlarni boshqa `file`ga yozish kerak bo‘lsa, `-n`ni qo‘shmang.

`exit status` ham ma'noli. `grep` odatda kamida bitta mos satr topsa `0`, birorta mos satr topmasa `1`, `file` yo‘qligi kabi bajarish xatosida `2` qaytaradi. Ekranda hech narsa chiqmaganda, qidirilgan matn yo‘qmi yoki `file` nomi xatomi — `exit status` orqali farqlang.

5.4 Tail bilan oxirgi satrlarni o‘qish

```
tail -n 5 incident.log
```

`tail -n 5` `file`ning oxirgi besh satrini asl tartibida `stdout`ga chiqaradi. Bu yerda `-n` satrlar sonini belgilaydi; `grep -n`dagi satr raqamini ko‘rsatish bilan ma'nosi boshqa. Faqat `option` harfini yodlash o‘rniga, har bir `command`ning manualini tekshiring.

`tail -f` `file`ga keyin qo‘shiladigan yangi satrlarni ham davomli ko‘rsatadi. To‘xtatish uchun `Ctrl+C`ni bosing.

5.5 Redirection yordamida natijani filega yozish

```
grep 'ERROR' incident.log > errors.txt
tail -n 5 incident.log > recent.txt
```

`>` — `shell` `stdout`ning manzilini `file`ga almashtiradigan `redirection`. O‘ng tomondagi `file` oldindan mavjud bo‘lsa, `command` ishlashidan avval u bo‘shatiladi va yangi natija bilan yoziladi. Shuning uchun kirish va chiqish uchun aynan bir `file`ni ko‘rsatish xavfli: `command` eski ma'lumotni o‘qishga ulgurmasidan mazmun yo‘qolishi mumkin.

```
# Xavfli misol. Bajarmang.
grep 'ERROR' incident.log > incident.log
```

`>>` ma'lumotni `file` oxiriga qo‘shadi. Uni bir necha marta ishlatsangiz, ayni satrlar takrorlanishi mumkin. Har safar mazmunni qayta yaratish kerak bo‘lsa, `>`dan foydalaning.

Odatda `>`ning o‘zi `stderr`ni shu `file`ga yo‘naltirmaydi. `2>` `stderr` uchun boshqa chiqish `file`ini ko‘rsatadi.

5.6 Pipe oraliq file yaratmasdan uzatadi

```
grep 'ERROR' incident.log | tail -n 5
```

`|` — `pipe`: chapdagi `command`ning `stdout`ini o‘ngdagining `stdin`iga to‘g‘ridan-to‘g‘ri ulaydi. Natijani diskdagi oraliq `file`ga yozmasdan bosqichma-bosqich saralash mumkin. Chapdagi `command`ning `stderr`i odatda bu `pipe`ga uzatilmaydi.

Bashning odatdagi sozlamasida butun `pipeline`ning `exit status`i oxirgi `command`ning natijasiga teng. Demak, chap tomon xato qilsa ham o‘ng tomon muvaffaqiyatli bo‘lsa, umumiy natija muvaffaqiyatli ko‘rinishi mumkin. Aniq avtomatik skriptlarda `set -o pipefail` kabi sozlamalarni ko‘rib chiqish kerak.

5.7 Quoting, o‘zgaruvchilar va command substitution

shell commandni boshlashdan avval kiritilgan matndagi maxsus yozuvlarni kengaytiradi.

```
name='incident.log'
printf '%s\n' "$name"
```

- Yakka qo‘shtirnoq '...': ichidagi belgilar deyarli aynan yozilganicha qabul qilinadi. $HOME kabi o‘zgaruvchi kengaytirilmaydi.
- Qo‘sh qo‘shtirnoq "...": bo‘sh joyli matnni bitta argument sifatida saqlaydi, lekin $HOME va $(...) kabi ifodalarni kengaytiradi.
- Qo‘shtirnoqsiz yozish: bo‘sh joy argumentni kutilmagan joyda ajratishi yoki wildcard kengayishi mumkin. O‘zgaruvchilarni odatda "$name" kabi qo‘sh qo‘shtirnoq bilan o‘rang.

$(command) — command substitution: ichidagi command bajariladi va uning stdout natijasi shu joyga matn sifatida qo‘yiladi.

```
current_user="$(id -un)"
printf 'USER=%s\n' "$current_user"
```

5.8 && faqat chap tomon muvaffaqiyatli bo‘lsa davom etadi

```
cd /var/log && pwd
```

&&ning o‘ngidagi command faqat chapdagi command exit status 0 bilan yakunlanganda bajariladi. Bu directoryga o‘tish muvaffaqiyatsiz bo‘lgach, noto‘g‘ri joyda fileni tahrirlash yoki o‘chirishning oldini oladi. Lekin bir satrga juda ko‘p bosqichni yig‘ish xato qayerda yuz berganini aniqlashni qiyinlashtiradi.

Bob yakunidagi savollar

1-savol. grep ERROR incident.log > errors.txtda errors.txtni grepmi yoki shell ochadimi?

2-savol. > va >> qanday farq qiladi? Ularni adashtirsangiz nima bo‘ladi?

Bob yakunidagi javoblar

1-savol

Javob: `shell` ochadi. U `errors.txt`ni ochib, `grep`ning `stdout`iga ulaydi. Shundan keyin `grep`ni ishga tushiradi.

2-savol

Javob: `>` mavjud mazmunning ustidan yozadi; `>>` esa oxiriga qo‘shadi. `>`ni noto‘g‘ri ishlatish eski ma’lumotni yo‘qotadi. `>>`ni keragidan ortiq takrorlash bir xil mazmunni ko‘paytiradi.

Foydalanilgan manbalar

- Ubuntu 24.04 amaliy muhitidagi `man grep`, `man tail`, `man bash` (nashrdan oldin versiyalar tekshiriladi)
- GNU Bash Reference Manual (2026-09-18 kuni veb-matnni yuklashda timeout bo‘lgan; amaliy muhitdagi manual asosiy tekshiruv manbasi)

6-bob. User, group va login

6.1 User turlari va vazifalari

Linuxda odam ishlatadigan hisoblargina emas, OS va service processlarini ishlatish uchun hisoblar ham bor. Linux ichkarida userni UID (user ID), groupni esa GID (group ID) raqami bilan ajratadi. Avval ularning vazifasi va huquqlari farqini ko‘raylik.

Turi	Asosiy vazifasi	Interaktiv login bilan aloqasi
Administrator root	Butun tizimni boshqaradigan maxsus user. UID qiymati 0.	Hisob mavjud, lekin Ubuntuda odatda root paroli bilan bevosita login o‘chiriladi. Ruxsat berilgan user kerakli boshqaruv amalini sudo orqali bajaradi. sudo commandni administrator yoki boshqa user huquqi bilan bajarish mexanizmidir.
Oddiy user	Odamning o‘z ishini bajaradi. Masalan, ssm-user va jduops.	Sozlama va authentication usuli ruxsat bersa, interaktiv login qiladi. Oddiy user bo‘lish avtomatik ravishda sudo huquqi bor degani emas.
service uchun user	Veb-server kabi processni faqat kerakli huquqlar bilan ishlatadi. Masalan, jduweb.	Odatda interaktiv login talab qilinmaydi. Ko‘pincha login shelli /usr/sbin/nologin qilib qo‘yiladi, ammo bu har bir service hisobiga majburiy qoida emas.

Jadval Linuxning ichida userlar aynan uch turga qat’iy bo‘linganini anglatmaydi. Bu ularning vazifasini tushunish uchun qulay tasnif. Aslida qaysi amal mumkinligi hisob sozlamalari, grouplar, har bir filening permissioni, sudo va SSH sozlamalari bilan belgilanadi. root hisobining mavjudligi bilan unga bevosita login qilish imkoniyati ham alohida masalalardir.

ssm-user kabi nomlar odam uchun qulay belgidir. file va directory egasi haqidagi ma’lumot tizim ichida raqamli UID/GID sifatida saqlanadi.

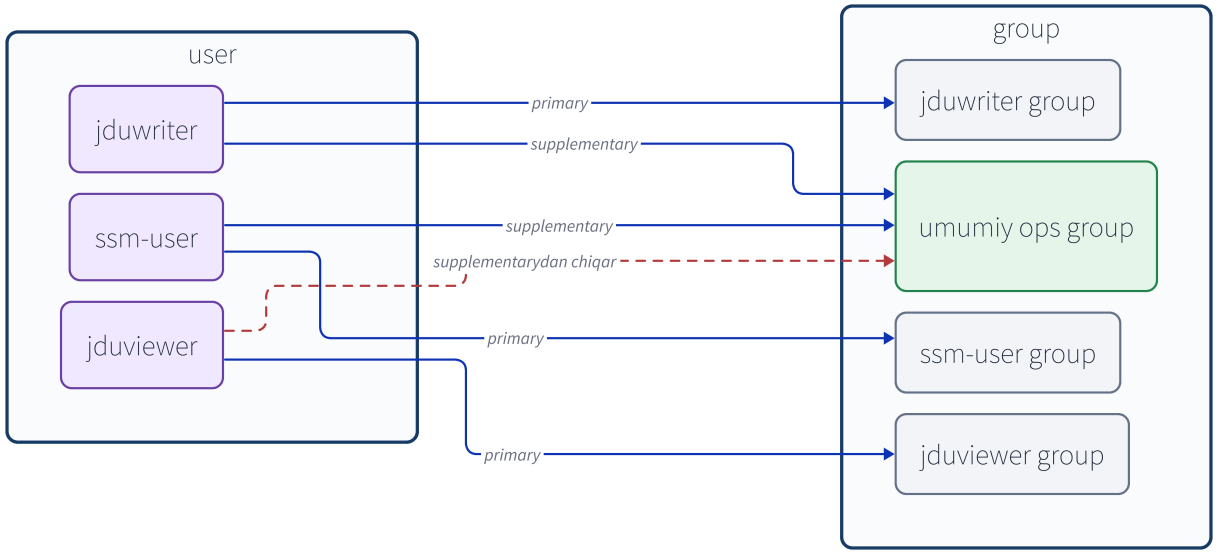
```
id
id jduops
getent passwd root
getent passwd jduops
getent passwd jduweb
getent group ops
```

id hozir ishlayotgan userning UID, primary group va supplementary grouplarini ko‘rsatadi. Argument sifatida user nomini bersangiz, uning ro‘yxatdan o‘tgan ma’lumotlarini tekshirasiz. getent passwd USER user nomi, UID, primary group GID, home directory va login shellini ko‘rsatadi. Masalan, oxirgi maydon /usr/sbin/nologin bo‘lsa, bu hisob odatdagi interaktiv shell uchun ishlatilmaydi. getent faqat mahalliy filelarni emas, tizim sozlagan nom xizmatlarini ham so‘raydi. Korporativ muhitda LDAP yoki Active Directory hisoblari ham natijaga kirishi mumkin.

6.2 Primary group va supplementary group

Har bir user uchun bittadan primary group belgilanadi. Ubuntuda yangi user yaratilganda, odatda uning nomi bilan bir xil maxsus group yaratiladi va primary group qilib qo‘yiladi. Lekin bu o‘zgarmas majburiy qoida emas. Kerak bo‘lsa, masalan usermod -g bilan o‘zgartirish mumkin.

Bir user o‘z primary groupidan tashqari bir nechta supplementary group a’zosi bo‘la oladi. Jamoaviy file yoki directoryga kirish berish uchun primary groupni o‘ylamasdan almashtirish o‘rniga, shu maqsad uchun tuzilgan supplementary groupga userni qo‘shish ma’qul.



6-1-rasm. Har bir userning primary groupi va umumiy ishlash uchun qo‘llanadigan supplementary grouplar o‘rtasidagi munosabat.

supplementary groupga qo‘shish va undan chiqarish uchun quyidagi commandlardan foydalaniladi.

```
sudo usermod -aG ops jduops
sudo gpasswd -d jduviewer ops
```

usermod -aGdagi -a (append) eski a’zoliklarni saqlagan holda yangisini qo‘shadi. Agar faqat -G ishlatib, -a tushirib qoldirilsa, ko‘rsatilmagan barcha eski supplementary grouplardan user chiqarilishi mumkin. O‘zgartirgach, id USER bilan ma’lumotni qayta tekshiring.

getent group ops oxirida ko‘rinadigan a’zolar ro‘yxati shu groupga supplementary group sifatida kirganlarni tekshirishga yordam beradi. Lekin ops primary group bo‘lgan userning nomi bu ro‘yxatda ko‘rinmasligi mumkin. Barcha a’zolarni aniqlash uchun id USER kabi vositalar bilan ham tekshiring.

6.3 Ro‘yxatdagi ma’lumot va ishlayotgan process huquqlari

group a’zoligi o‘zgartirilganda, oldindan ishlab turgan shell processining huquqlari va undagi grouplar ro‘yxati avtomatik yangilanmaydi. Yangi ma’lumotni olish uchun qayta login qilish yoki yangi session boshlash kerak.

```
id jduops
sudo su - jduops
id
```

Birinchi qatordagi id jduops tizimda ro‘yxatdan o‘tgan hisobni tekshiradi. su -dan keyingi id esa yangi login shell processi haqiqatda olgan huquqlarni ko‘rsatadi. kernel permissionni statik ro‘yxat yozuviga emas, amalni bajarayotgan processning UID/GID qiymatlariga qarab tekshiradi.

6.4 su bilan boshqa userning shellini ochish

su boshqa user huquqi bilan command bajarishga imkon beradi. Bu yerda boshqa user sifatida ishlashni davom ettirish uchun uning shellini ochishni o‘rganamiz.

```
su - jduops
id -un
pwd
exit
id -un
```

su - jduops jduops huquqi bilan login shellni boshlaydi. - belgisi shu user odatdagidek login qilgandagi muhitga yaqin muhitni yaratadi: uning home directorysiga o‘tadi va home directory, shell kabi muhit o‘zgaruvchilarini boshlang‘ich holatga keltiradi. Almashgandan keyin id -un bilan user nomini, pwd bilan ish directorysini tekshiring.

Asl userning ro‘yxatdagi ma’lumotlari o‘zgarmaydi. Asl shell ichida boshqa userning yangi shelli ochiladi. exit uni tugatib, oldingi shellga qaytaradi. Oxirgi id -un asl userga qaytganingizni tasdiqlaydi.

Oddiy user bevosita su - USERni ishlatsa, odatda maqsadli userning paroli so‘raladi. Yuqoridagi misol shu parol orqali authentication qilish mumkin bo‘lgan muhit uchun berilgan. Darsdagi Ubuntuda keyingi bo‘limda tushuntiriladigan sudo su - USERdan foydalanamiz. Paroli sozlanmagan userga bevosita su bilan o‘tish muvaffaqiyatli bo‘lishi shart emas.

su substitute user deb tushuntiriladi. Inglizcha substitute «o‘rniga qo‘yish, almashtirish» degani. Bu yerda command bajaradigan user boshqa user bilan almashtiriladi. Switch user, ya’ni «userni almashtirish», amalni eslab qolishga yordam beradigan ifodadir.

6.5 sudo bilan commandni boshqa user huquqi orqali bajarish

sudo sozlamada ruxsat olgan userga commandni administrator (root) yoki boshqa user huquqi bilan bajarishga imkon beradi. Oddiy account mavjudligining o‘zi sudo huquqini bermaydi. sudoers kabi sozlamalar kim, qaysi user huquqi bilan, qaysi commandlarni bajara olishini belgilaydi. Boshqaruv huquqi zarur amallar bilan cheklanishi kerak.

Odatdagi sozlamada maqsadli user ko‘rsatilmasa, command root huquqi bilan bajariladi. Misolda faqat sudo qo‘shilgan id -un root sifatida ishlaydi. Keyingi id -un asl shellda bajarilib, asl user nomini ko‘rsatadi.

```
sudo id -un
id -un
```

Odatdagi sozlamada parol talab qilinsa, sudo uni ishga tushirgan userning o‘z parolini so‘raydi. Bu maqsadli user parolini so‘raydigan sudan farq qiladi. Authentication usuli sozlamaga bog‘liq, parol so‘ramaydigan sozlama ham bo‘lishi mumkin.

Bitta commandni boshqa user huquqi bilan bajarish uchun sudo -u USER -- COMMAND ishlatiladi. -u USER maqsadli userni bildiradi, -- esa sudo optionlari tugaganini ko‘rsatadi. Bunda interaktiv shell ochilmaydi. Command tugagach, asl shellda ishlashni davom ettirasiz.

```
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

Bu jduweb huquqi bilan fileni o‘qish mumkinligini tekshiradi. test -r muvaffaqiyatli bo‘lsa ham hech narsa ko‘rsatmaydi. Darhol keyingi echo $? o‘qish mumkin bo‘lsa 0, mumkin bo‘lmasa 1 ko‘rsatadi. Orada boshqa command bajarilsa, tekshirilayotgan exit status o‘zgaradi. root fileni o‘qiy olishi service user ham o‘qiy olishini isbotlamaydi.

Darsda boshqa userning shellida bir necha amalni davom ettirish uchun quyidagi birikma ishlatiladi.

```
sudo su - jduops
id
pwd
exit
id -un
```

Bu bitta command nomi emas: sudo su - jduopsni ishga tushiradi. Odatdagi sozlamada sudo ruxsatni tekshirib, suni root sifatida bajaradi. Keyin su jduopsning login shellini ochadi. exit bilan qayting va id -un orqali asl userni tekshiring.

sudo superuser do, ya’ni «administrator sifatida bajarish» deb tushuntiriladi. Uni suning substitute user ifodasi bilan birga eslab qolish amallarni ajratishga yordam beradi. Hozirgi sudoda -u orqali rootdan boshqa user ham ko‘rsatilishi mumkin. Nomlari o‘xshash bo‘lsa ham, su va sudo alohida commandlardir. Boshqa userning shellini ochib ishlash va ruxsat berilgan commandni boshqa user huquqi bilan bajarishni ajratib qo‘llang.

6.6 Service user orqali process huquqlarini ajratish

service uchun alohida user uning process huquqlarini odam ishlatadigan hisoblar va boshqa servicelardan ajratadi. Masalan, veb-server processi jduweb sifatida ishlasa, permission jduwebning UID va GID qiymatlariga qarab tekshiriladi. Shunda faqat servicega kerakli filelarni o‘qish huquqini berish mumkin.

```
getent passwd jduweb
id jduweb
```

/usr/sbin/nologin odatdagi login shelli o‘rnida ishga tushadigan, interaktiv loginni rad etib tugaydigan programdir. Shunday sozlama bo‘lsa ham hisob mavjud. systemd service sozlamasida ko‘rsatilgan user huquqi bilan process boshlay oladi. Odam shu hisob orqali interaktiv login qila olmasligi bilan service processining shu UID nomidan filelarga murojaat qilishi boshqa-boshqa masaladir.

6.7 Account, authentication va authorization

account tizimda ro‘yxatdan o‘tgan user haqidagi ma’lumotdir. U user nomini UID, primary group, home directory va login shelli bilan bog‘laydi. “ssm-user accounti bor” degani server shu nomni taniydi, xolos. Ulangan odam haqiqatda shu hisobdan foydalanishga haqli ekanini hali isbotlamaydi.

authentication ulangan tomon da’vo qilgan accountning haqiqiy foydalanuvchisi ekanini tekshirishdir. Parol usulida maxfiy parol, SSH public key authenticationda tegishli private keyga egalik tekshiriladi. Ochiq kalit bilan ishlaganda, ulanish maqsadidagi userning ~/.ssh/authorized_keys fileiga public key kiritiladi. private keyning o‘zi serverga yuborilmaydi.

authorization muayyan tomon aniq bir amalni bajara olishini hal qilishdir. SSHda hisob va server sozlamalari ulanish mumkinligini ham belgilaydi. Ulangandan keyin fileni o‘qish, yozish yoki commandni administrator huquqi bilan bajarish har biri alohida qoida bilan tekshiriladi. Login muvaffaqiyatli bo‘lishi

barcha amallarga ruxsat degani emas.

SSH orqali ssm-user sifatida ulanishni shu uch tushunchaga ajrating.

1. account: ulanish tomonida ssm-user ro‘yxati bor. Tizim uning UID, home directory va boshqa ma’lumotlarini topa oladi. Hisob bo‘lmasa, odatdagi sessionni shu user nomidan boshlay olmaydi.
2. authentication: ulanuvchi private keydan foydalanadi; ulanish tomonidagi tizim u ssm-user uchun ro‘yxatdagi public keyga mosligini tekshiradi. Kalit mos bo‘lmasa, account mavjud bo‘lsa ham tekshiruvdan o‘tilmaydi.
3. authorization: SSH sozlamalari ulanishga yo‘l qo‘ygach ham, yangi process ssm-userning UID va guruhlariga mos huquqlar bilan cheklanadi. Masalan, boshqa usergagina o‘qish mumkin bo‘lgan fileni o‘qiy olmaydi. sudo huquqi ham alohida sozlama bilan belgilanadi.

Bu tushunchaviy ajratishdir; SSH programining ichki amallari aniq vaqt tartibini ifodalamaydi. SSH kaliti to‘g‘ri bo‘lsa ham, SSHga oid kirish cheklovi yoki account sozlamasi sababli interaktiv shell ochilmasligi mumkin. service user esa SSH bilan login qilmasdan, systemd tomonidan belgilangan UID bilan process boshlashi mumkin. Account mavjudligi, login huquqi va processning biror amalni bajarish huquqi alohida qarorlardir.

Bob yakunidagi savollar

1-savol. Primary group va supplementary group farqi nima? Umumiy directory misolida tushuntiring.

2-savol. Group ro‘yxati o‘zgarsa ham ochiq shell nega darhol yangilanmaydi? Nimani qayta boshlash kerak?

3-savol. Nima uchun sudo -u jduweb -- test -r FILE o‘rniga root nomidan cat FILE ishlatib bo‘lmaydi?

Bob yakunidagi javoblar

1-savol

Javob: primary group har bir userga majburiy belgilanadigan asosiy group. supplementary group qo‘shimcha a’zolikdir. Bir necha user umumiy directory bilan ishlashi uchun alohida group tuzilib, kerakli userlar unga qo‘shiladi.

2-savol

Javob: Ishlab turgan shell ishga tushganda olgan UID va group huquqlarini saqlaydi. Yangi grouplar ro‘yxatini olish uchun qayta login qilish yoki yangi session boshlash kerak.

3-savol

Javob: root fileni o‘qiy olishi jduweb processida ham o‘qish permissioni borligini bildirmaydi. sudo -u jduweb aynan jduweb huquqi bilan o‘qish mumkinligini tekshiradi.

Foydalanilgan manbalar

- Ubuntu Server documentation, User management (2026-09-24 kuni tekshirilgan; root, oddiy va tizim userlari)
- Ubuntu Server documentation, OpenSSH server (2026-09-24 kuni tekshirilgan; ochiq kalit bilan authentication va authorized_keys)
- Ubuntu 24.04 man pages, adduser(8), systemd.exec(5) (2026-09-24 kuni tekshirilgan; service user va processni ishga tushirish)
- Linux man-pages, passwd(5), credentials(7) (2026-09-24 kuni tekshirilgan; UID/GID, login shelli va process huquqlari)
- Ubuntu 24.04 amaliy muhitidagi man id, man getent, man usermod, man gpasswd, man su, man sudo (nashrdan oldin tekshiriladi)
- util-linux: su(1) (2026-10-13 tekshirilgan: substitute user, login shell va authentication)
- Ubuntu Server: Welcome to the terminal (2026-10-13 tekshirilgan: superuser do)
- Sudo: sudo manual (2026-10-13 tekshirilgan: ruxsat, maqsadli user va authentication)

7-bob. Permission va umumiy directory

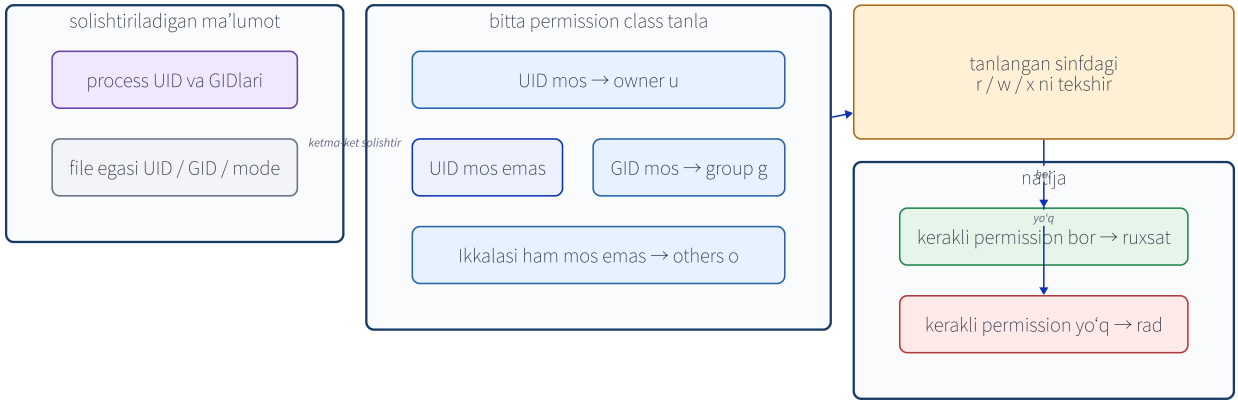
7.1 Permission kimga qaysi amal mumkinligini bildiradi

Linuxdagi asosiy permission uchta sinf uchun belgilanadi: egasi bo‘lgan user (u: owner user), egasi bo‘lgan group (g: owner group) va boshqalar (o: others). Har bir sinfga o‘qish (r: read), yozish (w: write) va bajarish (x: execute) huquqlari berilishi mumkin.

```
-rw-rw-r-- 1 root ops ... README.txt
drwxrwsr-x ... root ops ... /srv/jdu-share
```

Birinchi belgi obyekt turini bildiradi: - oddiy file, d directory. Keyingi to‘qqiz belgini egasi, group va boshqalar uchun uchtadan ajratib o‘qing.

kernel amalni bajarmoqchi bo‘lgan processning UID/GID huquqlarini va file metadatasini solishtiradi. Muhim jihat: egasi, group va boshqalarga tegishli barcha huquqlar qo‘shib hisoblanmaydi. process file egasi bo‘lsa faqat owner qatori; egasi bo‘lmay, tegishli groupga kirsa faqat group qatori; ikkalasiga ham kirmasa others qatori qo‘llanadi.



7-1-rasm. Process huquqlariga ko‘ra u/g/o sinfi tanlanadi, keyin shu sinfdagi rwx belgilari amal mumkinligini aniqlaydi.

7.2 Rwx file va directory uchun turlicha ma’noga ega

Huquq	Oddiy file	directory
r	Ichidagi ma’lumotni o‘qish	Ichidagi nomlar (file va directory nomlari) ro‘yxatini o‘qish
w	Mazmunni o‘zgartirish yoki qo‘shish	Ichida yangi nom yaratish yoki mavjud nomni o‘chirish
x	program sifatida bajarish	Ichidagi nomga o‘tish, ko‘rsatilgan pathga yetish (traverse)

directoryga yozish (w) huquqi bo‘lsa, uning ichidagi filening o‘zida yozish huquqi bo‘lmasa ham, ayrim hollarda uni o‘chirish mumkin. Chunki fileni o‘chirish directory ichidagi nomlar ro‘yxatini o‘zgartiradi. Aksincha, filening o‘zida o‘qish (r) bo‘lsa ham, yuqori directorydan o‘tish (x) huquqi bo‘lmasa, unga yetib borib o‘qib bo‘lmaydi. namei -l PATH butun yo‘ldagi directorylarning permissionini alohida ko‘rsatadi.

7.3 Sakkizlik sanoq tizimidagi modeni o‘qish

permissionning sonli yozuvi — sakkizlik mode — r=4, w=2, x=1 qiymatlari yig‘indisidir.

- 6 = 4+2 = rw-
- 7 = 4+2+1 = rwx
- 5 = 4+1 = r-x

Shunday qilib, oddiy file uchun 664 rw-rw-r--, directory uchun 775 rwxrwxr-x bo‘ladi.

```
sudo chmod 664 /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
```

777 hamma huquqlarni, jumladan others uchun o‘qish, yozish va bajarishni ochadi. Bu faqat umumiy group a’zolariga huquq berish maqsadiga mos emas.

7.4 Chown va chmod

chown egasi bo‘lgan user va groupni, chmod esa permission modeni o‘zgartiradi.

```
sudo chown root:ops /srv/jdu-share
sudo chown root:ops /srv/jdu-share/README.txt
sudo chmod 2775 /srv/jdu-share
sudo chmod 664 /srv/jdu-share/README.txt
```

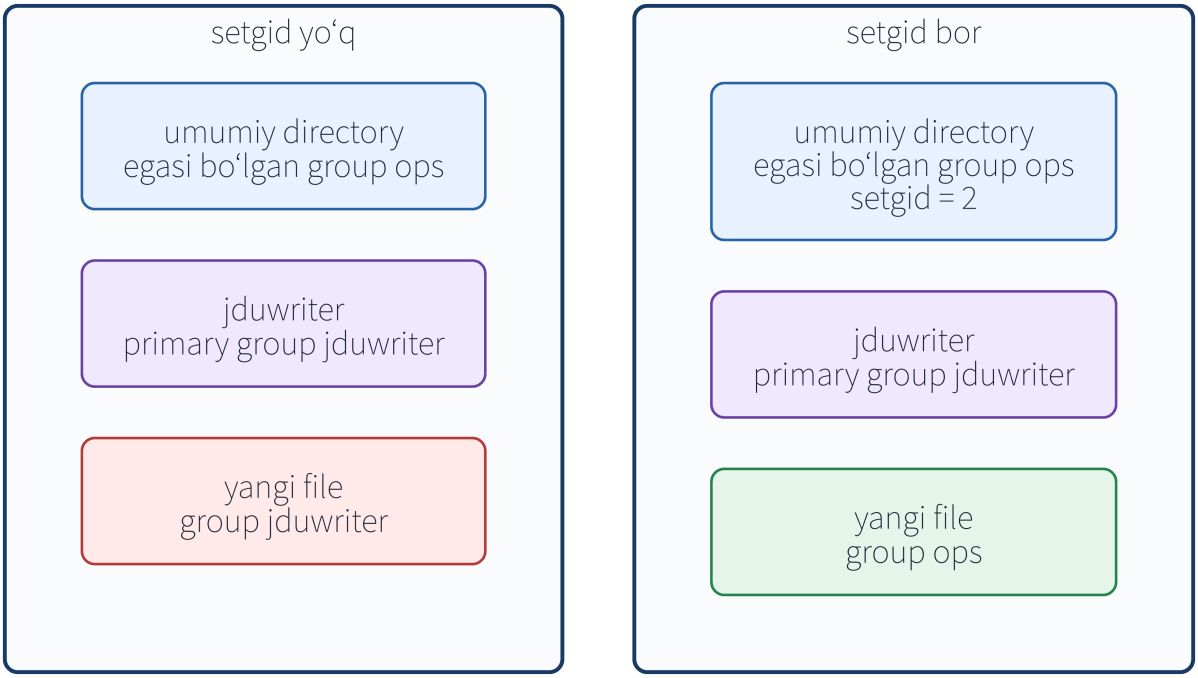
directory egasi yoki permissionini almashtirish uning ichida oldindan mavjud barcha file va kichik directorylarning atributini avtomatik o‘zgartirmaydi. -R bilan ichidagi hamma obyektga rekursiv ta’sir qilish mumkin, lekin kutilmagan filelar ham o‘zgarib ketishi xavfi bor.

```
stat -c '%U:%G %a %n' /srv/jdu-share /srv/jdu-share/README.txt
```

7.5 Setgid directory qanday muammoni hal qiladi?

Odatda yangi yaratilgan filening egasi bo‘lgan group yaratuvchi processning primary groupi bo‘ladi. Bir necha user bitta umumiy directoryda ishlasa, yangi filelar turli grouplarga tegishli bo‘lib qolishi mumkin. Natijada boshqa a’zo uni tahrirlay olmaydi.

directoryga setgid biti (Set Group ID) qo‘yilsa, shu directory ichida yaratiladigan yangi file va kichik directorylar odatda yaratuvchining primary groupini emas, yuqori directoryning egasi bo‘lgan groupni meros oladi. Masalan, /srv/jdu-share root:ops 2775 bo‘lsa, yangi filening groupi odatda ops bo‘ladi.



7-2-rasm. Setgid yangi obyektning groupini yuqori directory groupi bilan bir xil qiladi. U file egasi, mazmuni yoki butun permission modeni nusxalamaydi.

Sakkizlik mode boshidagi 2 setgid bitidir. Belgilar bilan yozilganda groupning bajarish joyida x o‘rniga s chiqadi.

```
drwxrwsr-x root ops /srv/jdu-share
```

Agar groupga bajarish (x) huquqi berilmagan bo‘lsa-yu, setgid bor bo‘lsa, katta S ko‘rinadi. Umumiy directoryda groupga odatda rwx (shu jumladan x) beriladi, shuning uchun kichik s chiqadi.

7.6 Umask yangi modega ta’sir qiladi

setgid group meros bo‘lishini boshqaradi; u yangi filening butun permission modeini majburan 664 qilmaydi. Yangi filening haqiqiy permissioni program so‘ragan asosiy modedan umask cheklagan bitlar chiqarilgach aniqlanadi.

```
umask
touch /srv/jdu-share/sample.txt
stat -c '%U:%G %a %n' /srv/jdu-share/sample.txt
```

Masalan, odatdagi file 666 asosida yaratilsa va umask 002 bo‘lsa, natija 664 (rw-rw-r--) bo‘ladi. Ammo program boshidan boshqa mode so‘rashi mumkin. umask faqat yangi yaratish paytida ta’sir qiladi; oldindan mavjud filening permissionini keyin o‘z-o‘zidan o‘zgartirmaydi.

7.7 Ruxsat va rad etishni haqiqiy user bilan sinash

Faqat metadatani ko‘rish bilan cheklanmay, kerakli user huquqi bilan amalni sinab ko‘ring.

```
sudo -u jduviewer -- test -r /srv/jdu-share/README.txt
echo $?
sudo -u jduviewer -- test -w /srv/jdu-share
echo $?
```

exit status 0 shart bajarilganini (muvaffaqiyat), 1 bajarilmaganini bildiradi. directory ichida yangi file yaratish unga yozish (w) va undan o‘tish (x) huquqlariga bog‘liq. Faqat mavjud filega yozish mumkinligini sinab, yangi file yaratish mumkin degan xulosaga kelmang.

Kirish rad etilsa, amalni bajargan processning user va groupini, qo‘llangan permission sinfini va yetishmagan huquqni ketma-ket tekshiring. Muammoni shunchaki 777 bilan “hal qilish” boshqa userlarga ham keraksiz huquq beradi. Faqat kerakli userga kerakli huquqni bering.

Bob yakunidagi savollar

1-savol. 2775 permission mode raqamlari nimani bildiradi?

2-savol. Setgid directoryda yangi file yaratilsa, nimasi yuqori directorydan meros bo‘ladi, nimasi bo‘lmaydi?

3-savol. Root file o‘qiy olishi service user ham o‘qiy olishini nima uchun isbotlamaydi?

Bob yakunidagi javoblar

1-savol

Javob: Birinchi 2 directory uchun setgidni bildiradi. Keyingi 7 egasining rwx, uchinchi 7 egasi bo‘lgan groupning rwx, oxirgi 5 othersning r-x huquqlaridir. directoryning belgili ko‘rinishi rwxrwsr-x.

2-savol

Javob: Yangi file odatda yuqori directoryning egasi bo‘lgan groupni meros oladi. Egasi bo‘lgan user yoki butun permission mode ko‘chirib olinmaydi. Yangi file modeiga uni yaratgan program so‘rovi va umask kabi sozlamalar ta’sir qiladi.

3-savol

Javob: root odatdagi file permission cheklovini chetlab o‘tib o‘qiy olishi mumkin. service user processi uchun esa uning UID va grouplariga tegishli permission tekshiriladi.

Foydalanilgan manbalar

- Linux man-pages, path_resolution(7) (2026-09-18 kuni tekshirilgan)
- Ubuntu 24.04 amaliy muhitidagi man chmod, man chown, man umask (nashrdan oldin tekshiriladi)

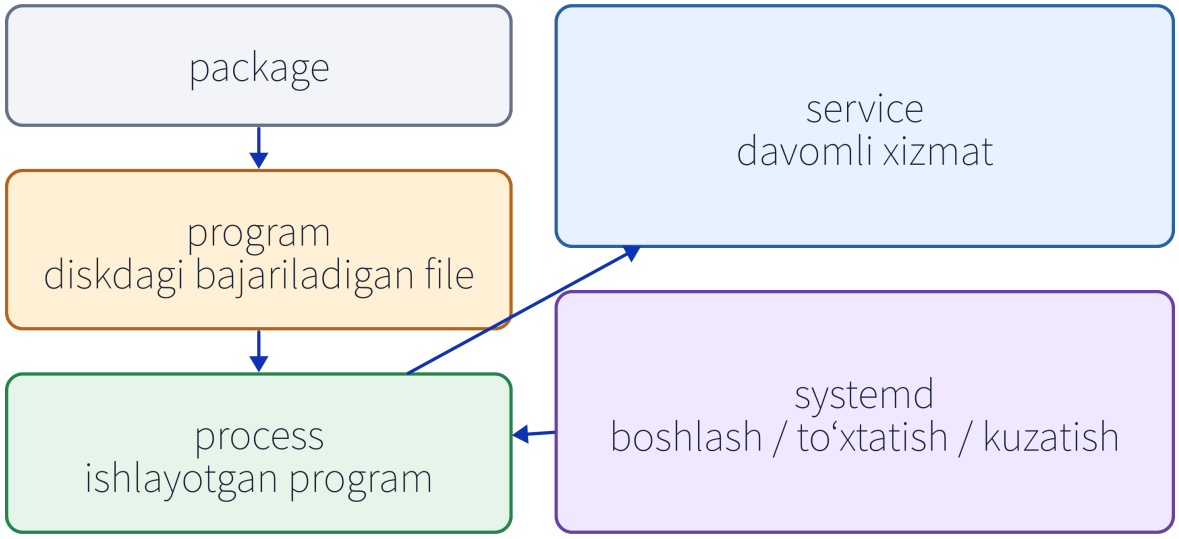
8-bob. Program, process, PID va signal

8.1 Program va process bir xil emas

program — saqlash qurilmasidagi ko‘rsatmalar va ularga tegishli ma’lumot filelari. process esa shu program xotiraga yuklanib, OS tomonidan bajarilayotgan holat. Bir programdan bir nechta process boshlash mumkin. Har biriga alohida PID (process ID) raqami beriladi.

package program va sozlama filelarini tarqatish hamda o‘rnatish birligidir. service esa fonda davomli imkoniyat beradigan ishni boshqarish birligidir. Quyidagi munosabatlarni farqlang.

```
Package o‘rnatiladi      → program filelari joylashtiriladi
Program ishga tushiriladi → process paydo bo‘ladi
Systemd unitni boshqaradi → service processini ishga tushiradi va kuzatadi
```



8-1-rasm. Saqlangan program, ishlayotgan process, boshqaruv birligi service va tarqatish birligi package alohida tushunchalardir.

8.2 PID va processlarning ota-bola munosabati

kernel har bir ishga tushirilgan processga musbat butun son — PID — beradi. process boshqa processni bola sifatida boshlashi mumkin. PPID (parent process ID) orqali ularning ota-bola munosabatini ko‘rish mumkin.

```
ps -e -o pid,ppid,user,stat,comm,args
ps -p 1234 -o pid,ppid,user,stat,comm,args
```

Bu yerdagi 1234 faqat misol. Haqiqiy PID har safar o‘zgarishi mumkin. process tugagach, ayni raqam keyin boshqa processga berilishi ham mumkin. signal yuborishdan oldin PID bilan birga user, command nomi (comm) va argumentlarni (args) tekshiring.

PID 1 tizim ishga tushganining boshidan ishlaydigan alohida boshqaruv processidir. Ubuntuda bu vazifani odatda systemd bajaradi.

8.3 Service boshqaradigan processni topish

Ubuntuda systemd servicelarning boshlanishi va holatini boshqaradi. serviceni tavsiflovchi sozlama unit deyiladi; service uniti nomi odatda .service bilan tugaydi. Sozlamada ko‘rsatilgan program bajarilganda, unga tegishli process ishlaydi. Boshqaruv obyekti bo‘lgan unit bilan hozir ishlayotgan process aynan bitta narsa emas.

systemctl systemd boshqaradigan unit holatini tekshiradi. Quyidagi misol ishlayotgan servicelar ro‘yxatini, bittasining holatini va asosiy processini ko‘rsatadi.

```
systemctl list-units --type=service
systemctl status systemd-journald.service --no-pager
```

statusdagi Main PID servicening asosiy process PIDidir. Ko‘rsatilgan raqamni yozib, ps -fp PIDdagi PID o‘rniga qo‘ying. Shunda service boshqaruv ma’lumoti va haqiqiy processni bog‘laysiz. ps processni, systemctl servicening boshqaruv holatini ko‘rsatadi. systemd boshqaradigan serviceni to‘xtatishda PIDga bevosita signal yubormasdan, systemctl stop SERVICE_NAMEni ishlating. To‘xtatishni systemd boshqaradi. Avtomatik ishga tushish 10-bobda ko‘riladi.

8.4 Foreground va background

terminaldan odatdagi command bajarilganda, shell uning foreground process sifatida tugashini kutadi. Shu paytda klaviatura kiritishi asosan shu processga boradi. Ctrl+C terminaldan foreground process groupga uzilish signalini yuboradi.

command oxiriga & qo‘yilsa, shell uni backgroundda boshlaydi va keyingi kiritishni qabul qiladi.

8.5 Signal processga xabar beradi

signal — OS yoki bir process boshqa processga hodisa yoki boshqaruv talabini asinxron tarzda bildiradigan mexanizm.

- TERM (SIGTERM): processdan ma’lumotni saqlash va resurslarni bo‘shatish kabi odatdagi yakunlash ishlarini bajarib, chiqishni so‘raydi. processni to‘xtatishda odatda avval ishlatiladi.
- KILL (SIGKILL): kernel processni majburan va zudlik bilan tugatadi. process bu signalni tutib, o‘zini tartibli yopishga ulgurmaydi. TERM kabi odatdagi signallar ishlamaganda oxirgi choradir.
- INT (SIGINT): masalan, terminaldagi Ctrl+C yuboradigan uzilish signali.

```
kill -TERM PID
```

Nomiga qaramay, killning vazifasi ko‘rsatilgan signalni processga yuborishdir. Masalan, pkill 1956da 1956 PID emas, process nomini izlash namunasi sifatida talqin qilinadi. PIDni aniq ko‘rsatish uchun kill, nom namunasi orqali tanlash uchun pkill ishlatiladi.

8.6 Process tugasa ham filelar qoladi

process tugatilsa ham saqlash qurilmasidagi bajariladigan program filei, unit filei va ma’lumot filelari saqlanib qoladi. Aksincha, diskdagi bajariladigan file o‘chirilsa ham, xotirada oldindan ishlayotgan process darhol to‘xtamaydi.

systemd boshqaradigan service processi tugagach qayta ishga tushishi mumkin. Bu unit fileidagi Restart= kabi sozlamaga bog‘liq. Shuning uchun processning tugashi bilan service doimiy to‘xtashi ham boshqa hodisalardir.

8.7 /proc/PID orqali joriy holatni ko‘rish

```
printf 'PID=%s\n' "$$"
cat "/proc/$$/cmdline" | tr '\0' ' '
printf '\n'
readlink -f "/proc/$$/cwd"
```

$$ hozirgi Bash shellining PIDidir. /proc/PID/cmdlinedagi argumentlar NUL (\0) belgisi bilan ajratiladi; tr ularni bo‘sh joyga almashtirib ko‘rsatadi. /proc/PID/cwd shu processning ish directorysiga ishora qiladigan ramziy link. process tugasa, unga mos /proc/PID ham yo‘qoladi.

/proc/PIDni o‘qish orqali saqlangan program fileini emas, hozir ishlayotgan process haqidagi ma’lumotni kuzatasiz.

Bob yakunidagi savollar

1-savol. Package, program, process va service farqi nima?

2-savol. Signal yuborishdan avval nima uchun PIDni ps bilan tekshirish kerak?

3-savol. TERM yuborgach service qayta ishga tushsa, unit fileda nimani tekshirish kerak?

Bob yakunidagi javoblar

1-savol

Javob: package program va sozlamalarni tarqatish-o‘rnatish birligi. program saqlangan ko‘rsatmalar. process shu programning ishlayotgan holati. service davomli imkoniyat beradigan, systemd kabi vosita boshqaradigan birlik.

2-savol

Javob: PID har ishga tushishda o‘zgarishi, tugagach boshqa processga qayta berilishi mumkin. Shu sababli PIDga qo‘shimcha ravishda user, command nomi va argumentlarni ham tekshirib, aynan kerakli process ekanini aniqlang.

3-savol

Javob: Restart= kabi avtomatik qayta ishga tushirish sozlamasini tekshiring. process tugaganidan keyin systemd sozlamaga binoan uni qayta boshlashi mumkin.

Foydalanilgan manbalar

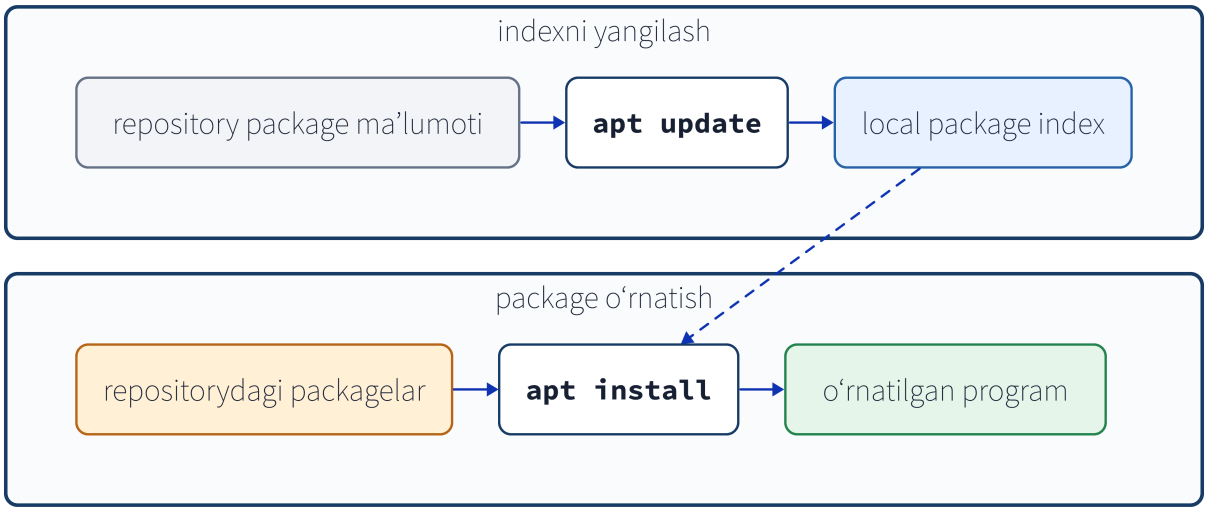
- Linux man-pages / Ubuntu 24.04dagi man ps, man kill, man proc (nashrdan oldin amalda tekshiriladi)
- GNU Bash Reference Manual, Special Parameters (2026-09-24 kuni tekshirilgan; $$ shell PIDidir)
- Ubuntu 24.04 man page, systemd.service(5) (2026-09-24 kuni tekshirilgan; Restart=)
- Ubuntu 24.04 man page, systemctl(1) (2026-09-24 kuni tekshirilgan; status va show)

9-bob. Package va dastur o‘rnatish

9.1 Nima uchun package kerak?

Amalda ishlatiladigan dasturiy ta’minot doim bitta bajariladigan fayldan iborat bo‘lmaydi. Unga umumiy kutubxonalar, sozlama fayllari namunalari, manual va ma’lumot fayllari ham kerak bo‘lishi mumkin. Package — shu o‘zaro bog‘liq fayllar, versiya ma’lumoti, bog‘liqliklar hamda o‘rnatish va o‘chirish skriptlarini birlashtiradigan tarqatish birligi.

Ubuntu asosan Debian package formatidan (.deb) foydalanadi va ularni APT (Advanced Package Tool) orqali boshqaradi. APT repositorydan mavjud packagelar haqidagi yangi ma’lumotni oladi, kerakli kutubxona bog‘liqliklarini aniqlaydi va packageni tizimga o‘rnatadi. Internetdan manbasi noma’lum skriptni root huquqi bilan bevosita ishga tushirish va ishonchli rasmiy repositorydagi tekshirilgan packageni o‘rnatish xavfsizlik jihatidan tubdan farq qiladi.



9-1-rasm. apt update mavjud packagelar ro‘yxatini yangilaydi; apt install esa package mazmunini olib o‘rnatadi.

9.2 Repository va package index

Repository — packagelar va ularga tegishli metadatani (versiya va bog‘liqlik ma’lumotlarini) tarqatadigan server. Ubuntu mahalliy package indexda qaysi repositoryda qaysi versiya borligi haqidagi ro‘yxatni saqlaydi.

```
sudo apt update
apt show cmatrix
sudo apt upgrade
```

apt update repositorydan mavjud packagelar ro‘yxatining yangi nusxasini oladi. Uning o‘zi o‘rnatilgan packagelarning mazmunini o‘zgartirmaydi. apt show cmatrix o‘rnatishdan oldin tavsif, versiya va bog‘liqliklarni ko‘rsatadi. apt upgrade yangilangan ro‘yxatga ko‘ra o‘rnatilgan packagelarni yangi versiyalarga o‘tkazadi. Davom etishdan oldin ekranda ko‘rsatilgan o‘zgarishlarni tekshiring. Bu Ubuntu tizimini yangi katta relizga almashtirish emas.

9.3 O‘rnatish va ishga tushishni tekshirish

```
sudo apt install cmatrix
command -v cmatrix
cmatrix
```

`sudo` huquqi kerak, chunki umumiy `package` ma’lumotlar bazasi va administrator himoyasidagi `/usr/bin` kabi joylar o‘zgaradi. O‘rnatilgach, `command -v` bilan `shell` bajariladigan `file`ni qaysi `path`da topishini tekshiring. `cmatrix`dan chiqish uchun `Ctrl+C`ni bosing.

`package` o‘rnatilgan bo‘lishi bilan uning `process`i hozir xotirada ishlashi boshqa holat. `command` tugagach ham `package` tizimda qoladi. Ba’zi `server` dasturlari o‘rnatilganda esa `service process`ini avtomatik boshlashi mumkin. Taxmin qilmang: haqiqiy holatni `systemctl` bilan tekshiring.

9.4 Versiya va file qaysi packagega tegishli ekanini aniqlash

```
dpkg-query -W -f='${Package} ${Version}\n' cmatrix
dpkg -S /usr/bin/apt
```

`dpkg-query` mahalliy `package` ma’lumotlar bazasidan o‘rnatilgan aniq versiyani oladi. `dpkg -S PATH` ko‘rsatilgan `file`ni qaysi `package` yetkazib berganini aniqlaydi. `command -v` “`shell` qaysi `path`dagi bajariladigan `file`ni ko‘ryapti?” degan savolga javob beradi. `dpkg -S` esa “bu `file` qaysi `package`ga tegishli?” degan boshqa savolga javob beradi. `/usr/bin/apt` — misol. Agar `command`ning e’lon qilingan `path`i `symbolic link` bo‘lsa, uning o‘zi `package`ga tegishli bo‘lmasligi mumkin. `readlink -f` bilan haqiqiy `path`ni topib, keyin shu `file`ni `dpkg -S` bilan tekshiring.

9.5 O‘chirish va tozalash

`apt remove` `package`ning asosiy qismini o‘chiradi. `apt purge` sozlama `file`lari bilan birga o‘chiradi. `apt autoremove` boshqa `package`larga endi kerak bo‘lmay qolgan bog‘liq `package`larni o‘chiradi. Ularning hammasi `apt upgrade`dan farq qiladi; bajarishdan oldin nimani o‘chirishi ko‘rsatilganini tekshiring.

Bob yakunidagi savollar

1-savol

`apt update`, `apt upgrade` va `apt install` vazifalarini farqlang.

2-savol

`command -v cmatrix` va `dpkg -S /usr/bin/apt` nimalarni tekshiradi?

3-savol

Package o‘rnatilgan, lekin unga tegishli process ishlamayotgan holatga misol keltiring.

Bob yakunidagi javoblar

1-savol

Javob: `apt update` mavjud `package`lar ro‘yxatini yangilaydi. `apt upgrade` o‘rnatilgan `package`larni yangi versiyaga yangilaydi. `apt install` ko‘rsatilgan `package`ni o‘rnatadi.

2-savol

Javob: `command -v cmatrix` `shell` bajaradigan `command`ning `path`ini topadi. `dpkg -S /usr/bin/apt` shu `file`ni yetkazib bergan `package`ni aniqlaydi. Agar e’lon qilingan `path` `symbolic link` bo‘lsa, avval haqiqiy `path`ni tekshiring.

3-savol

Javob: `cmatrix` o‘rnatilgach uni umuman boshlamaslik yoki ishga tushirib, `Ctrl+C` bilan tugatish. `package` tizimda qoladi, ammo `cmatrix` `process`i ishlamaydi.

Foydalanilgan manbalar

- Ubuntu Server documentation, Install and manage packages (2026-09-24 kuni tekshirilgan)
- Ubuntu Server documentation, Managing your software (2026-09-24 kuni tekshirilgan)
- Ubuntu 24.04 amaliy muhitidagi `man apt`, `man dpkg-query`, `man dpkg` (nashrdan oldin tekshiriladi)

10-bob. Systemd va service

10.1 Service nima?

service — foydalanuvchi har safar terminaldan qo‘lda ishga tushirmasa ham, OS fonda boshqaradigan va davomli funksiya beradigan program. Veb-server, SSH server, vaqtni sinxronlash va loglarni boshqarish bunga misol. Biroq tizimdagi har bir process service emas. Har bir service ham tarmoqda kutadigan portga ega bo‘lishi shart emas.

Ubuntu 24.04da systemd PID 1 sifatida boshlanadi va turli servicelarni markaziy tarzda boshqaradi. systemd ularning aniq ishini o‘zi bajarmaydi. U servicelarni boshlash, to‘xtatish, bog‘liqliklarini tartiblash, holatini kuzatish va tizim ishga tushganda avtomatik boshlashni boshqaradigan service managerdir.

10.2 Unit file — sozlama; process — joriy ish

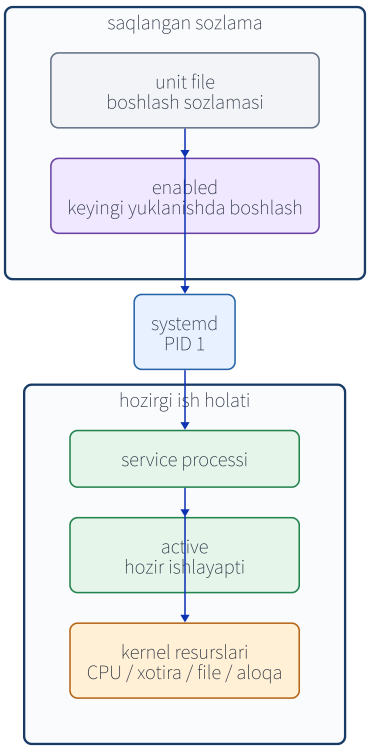
systemd resurslarni unit deb ataladigan boshqaruv birliklari orqali boshqaradi. serviceni belgilaydigan .service unit fileida qaysi bajariladigan fileni, qaysi user huquqi bilan va qaysi ish directorysida boshlash yoziladi.

```
[Service]
User=jduapp
WorkingDirectory=/srv/jdu-status
ExecStart=/usr/bin/python3 /srv/jdu-status/server.py
```

- User: service processi ishlaydigan user.
- WorkingDirectory: process boshlangandagi joriy ish directorysi.
- ExecStart: ishga tushiriladigan programning absolute pathi va argumentlari.

```
systemctl cat jdu-status.service
```

systemctl cat diskdagi unit fileini va uning ayrim sozlamalarini almashtiradigan drop-in yozuvlarini ko‘rsatadi. file o‘zgartirilib, hali daemon-reload bajarilmagan bo‘lsa, ko‘rsatilgan matn bilan systemd xotirasidagi sozlama farq qilishi mumkin.



10-1-rasm. Unit — sozlama, systemd — boshqaruvchi, service processi esa haqiqiy ishni bajaruvchi.

10.3 Active va enabled boshqa-boshqa holatlar

```
systemctl is-active jdu-status.service
systemctl is-enabled jdu-status.service
```

- active / inactive: service processi hozir xotirada ishlayaptimi yoki to‘xtaganmi.
- enabled / disabled: tizim keyingi safar yoqilganda shu unit avtomatik boshlanishga sozlanganmi.

Shunday qilib, “hozir active, ammo keyingi yuklanishda avtomatik boshlanmaydigan disabled” holati mumkin. “Hozir inactive, ammo keyingi yuklanishda avtomatik boshlanadigan enabled” holati ham mumkin. Darhol boshlash (start) va avtomatik boshlanishni yoqish (enable) ikki alohida amaldir.

```
sudo systemctl start jdu-status.service
sudo systemctl enable jdu-status.service
```

enable --now ikkala amalni birga bajaradi.

10.4 Start, stop, restart va reload

- start: to‘xtagan service processini boshlaydi.
- stop: ishlayotgan service processini to‘xtatadi.
- restart: to‘xtatib, yana boshlaydi; qisqa xizmat uzilishi yuz beradi.
- reload: processni to‘xtatmay, uning sozlamasini qayta o‘qishni so‘raydi. Hamma service buni qo‘llamaydi.

unit filei o‘zgargandan keyin systemdning o‘ziga yangi sozlamani o‘qitadigan systemctl daemon-reload va service processidan o‘z sozlamasini qayta o‘qishni so‘raydigan reload mutlaqo boshqa amallardir.

10.5 Unit sozlamasi va haqiqiy processni solishtirish

```
systemctl status jdu-status.service
systemctl show --property MainPID --value jdu-status.service
```

8-bobda Main PID orqali process topilgan edi. Endi unitdagi User=, ExecStart=, WorkingDirectory= qiymatlarini amaldagi process bilan solishtiring. systemctl status unit yuklanganini, ishlayotganini (active), Main PID va yaqindagi loglarni ko‘rsatadi. Natija uzun bo‘lsa pager ochilishi mumkin; undan q bilan chiqing. systemctl show kerakli property qiymatini alohida chiqaradi.

Main PIDni olgach, unit sozlamasi va ishlab turgan process mosligini tekshiring. Quyidagi 1234 faqat misol uchun: uni o‘z serveringizda ko‘rsatilgan Main PIDga almashtiring.

```
ps -p 1234 -o pid,user,comm,args
sudo readlink -f /proc/1234/cwd
```

unit sozlamasi	Haqiqiy processda tekshiriladigan joy
User=	psdagi USER ustuni
ExecStart=	psdagi args ustuni va /proc/PID/cmdline
WorkingDirectory=	/proc/PID/cwd ishora qiladigan manzil

10.6 Active bo‘lish funksiyaning to‘g‘riligini isbotlamaydi

systemd processni active deb ko‘rsatsa ham, foydalanuvchiga to‘g‘ri javob qaytmasligi mumkin. Masalan, kontent fileiga permission yetmasligi, kutilmagan IP manzilda kutishi (listen) yoki ilova ichida xato bo‘lishi mumkin. Keyingi bobda service boshqaruv holatidan tashqari process, socket, HTTP response va log ham tekshiriladi.

10.7 Systemctl status hamma processni ko‘rsatmaydi

systemctl statusni argumentsiz bajarish butun tizimning qisqa holati va process treeni ko‘rsatadi. Ayrim unit tafsilotlarini ko‘rish qiyin bo‘lishi mumkin. Muayyan service uchun uning unit nomini aniq kiriting.

```
systemctl status systemd-journald.service --no-pager
systemctl list-units --type=service 'systemd-*'
```

Foydalanuvchi terminalda boshlagan shell kabi ko‘plab processlar systemdning service uniti ostida ishlamaydi. Butun tizimdagi processlarni ko‘rish uchun ps, systemd boshqaradigan servicelar holati uchun systemctl ishlatiladi.

Bob yakunidagi savollar

1-savol

Unit file, systemd va service processi vazifalari nimasi bilan farqlanadi?

2-savol

Service active va disabled bo‘lishi bir vaqtda mumkinmi? Bu nimani anglatadi?

3-savol

Unit fileda belgilangan `WorkingDirectory` ishlab turgan processga qo‘llanganini `/proc` orqali qanday tekshirasiz?

Bob yakunidagi javoblar

1-savol

Javob: unit file ishga tushirish usuli kabi sozlamalarni saqlaydi. systemd shu sozlamalarga ko‘ra serviceni boshqaradi. service processi esa haqiqiy ishni bajaradigan ishlab turgan programdir.

2-savol

Javob: Ha. active hozir ishlayotganini, disabled esa keyingi yuklanishda avtomatik boshlanish yoqilmaganini bildiradi. Qo‘lda start qilingan service shunday bo‘lishi mumkin.

3-savol

Javob: systemctl show --property MainPID --value unit_name bilan asosiy PIDni toping. readlink -f /proc/PID/cwd shu processning hozirgi ish directorysini ko‘rsatadi. Natijani unit filedagi WorkingDirectory= bilan solishtiring.

Foydalanilgan manbalar

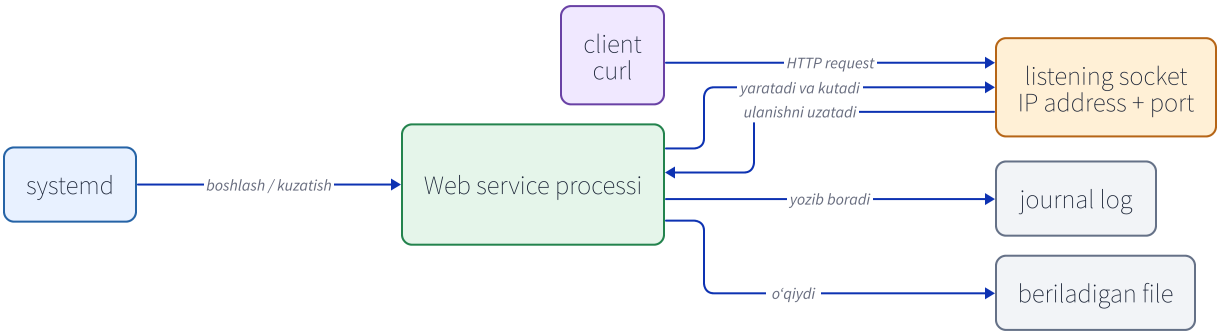
- Ubuntu 24.04 systemctl(1), systemd.exec(5), proc_pid_cwd(5) (2026-09-24 kuni tekshirilgan)
- Ochiq Lab v1.0.0dagi P4/M4 unit va checker (Labga xos qiymatlar uchun asosiy manba)

11-bob. Service holatini socket, HTTP va log orqali kuzatish

11.1 Service, process va socket munosabati

systemd unit sozlamasiga binoan service processini ishga tushiradi. Agar programga tarmoq aloqasi kerak bo‘lsa, system call orqali kerneldan socket yaratishni so‘raydi. kernel IP manzil, aloqa protocoli va port raqamiga qarab kelgan ma’lumotni tegishli processga yetkazadi.

“service o‘zi socket yaratadi” degandan ko‘ra, “service sifatida ishlayotgan program kernel bergan socket funksiyasidan foydalanib aloqa qiladi” deyish aniqroq. systemd processni boshqaradi va kuzatadi; tarmoqdagi barcha ma’lumotni uning o‘zi yuborib-qabul qilmaydi.



11-1-rasm. Boshqaruv (systemd), bajarish (process), aloqa (kernel va socket) hamda client requesti alohida vazifalardir. Bir service bir nechta socketga ega bo‘lishi yoki umuman tarmoqdan foydalanmasligi mumkin.

11.2 Protocol, IP address va port

Veb-aloqada keng ishlatiladigan transport protocollaridan biri TCP (Transmission Control Protocol). TCP ma’lumot yetganini tasdiqlash va uni tartib bilan yetkazish imkonini beradi. Ulanishni boshlash va qayta yuborishning tafsilotlari keyingi tarmoq bo‘limida o‘rganiladi.

IP address tarmoq interfaceini (aloqa qilinadigan hostni) aniqlaydi. port raqami esa ayni host ichidagi qaysi aloqa nuqtasiga, ya’ni qaysi processga ma’lumot yetkazilishini farqlaydi. port raqamining o‘zi program emas. kernel socketning kutish ma’lumotini port bilan solishtirib, ma’lumotni tegishli processga beradi.

- 127.0.0.1: IPv4dagi loopback address. Unga tashqi tarmoqdan bevosita yetib bo‘lmaydi; odatda faqat shu host ichidan ulanish mumkin.
- 0.0.0.0: shu hostning barcha mahalliy IPv4 manzillarida (barcha tarmoq interfacelarida) kutishni bildiradigan maxsus yozuv.
- [::]: barcha mahalliy IPv6 manzillarida kutishni bildiradi. OS sozlamasiga qarab IPv4 ulanishlarini ham qabul qilishi mumkin.

11.3 Ss bilan listening socketni o‘qish

ss (Socket Statistics) kernel boshqaradigan socketlarning batafsil holatini ko‘rsatadi.

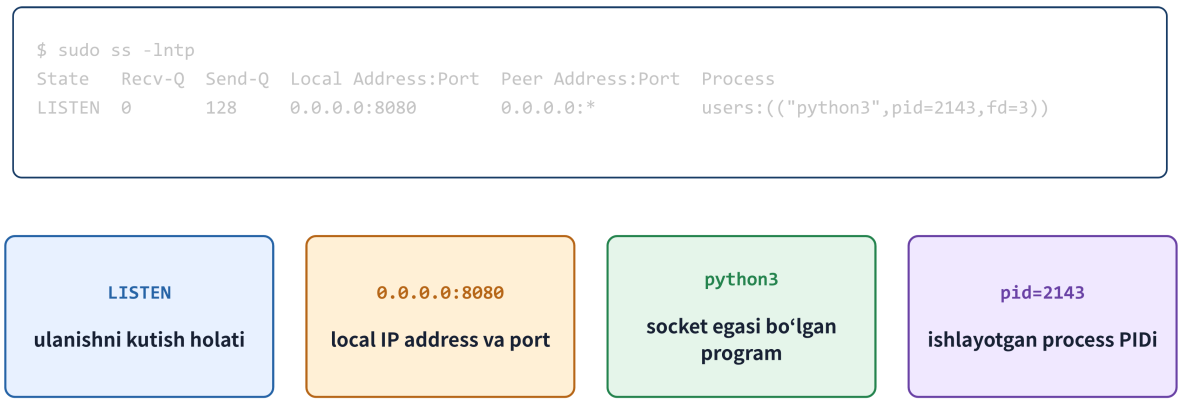
```
sudo ss -lntp | grep ':8081'
```

- `-l`: faqat kutish (`LISTEN`) holatidagi `socket`larni ko‘rsatadi.
- `-n`: `port`ni xizmat nomiga (masalan, http) almashtirmay, raqamligicha ko‘rsatadi.
- `-t`: faqat TCP `socket`larini ko‘rsatadi.
- `-p`: `socket`dan foydalanadigan `process` nomi va PIDini ko‘rsatadi. Ba’zan buning uchun administrator huquqi kerak.

Tushunchaviy chiqish namunasi:

```
State  Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0      5      127.0.0.1:8081  0.0.0.0:*      users:(("python3",pid=2451,fd=3))
```

Natijani avval holat (`State: LISTEN`), keyin kutish manzili va `port`i (`Local Address:Port: 127.0.0.1:8081`), so‘ng `process` (`pid=...`) tartibida o‘qing. Kutish manzili `127.0.0.1:8081` ekanini va ko‘rsatilgan PID `systemd`dagi `service` Main PIDiga mosligini solishtiring. Bu yerda `2451` faqat misol; o‘z muhitingizdagi haqiqiy PIDni o‘qing.



11-2-rasm. `ss -lntp` chiqishidagi satr holat, manzil, port va process ma’lumotiga ajratib o‘qiladi.

11.4 Client, server va HTTP

`server process` `socket` yaratib, `client` ulanishi va `request`ini kutadi. `client` `server socket`iga ulanib, `request` yuboradi. Amaliy muhitda Ubuntu `host`ining o‘zida ishlatiladigan `curl` `client`, `jdu-web.service` `process`i esa `server` vazifasini bajaradi.

```
curl -i http://127.0.0.1:8081/
```

`http://127.0.0.1:8081/` URLida `http` aloqa `protocol`i (`scheme`), `127.0.0.1` ulaniladigan `host`, `8081` `port`, `/` esa `request` qilinadigan `path`. `-i` `response body` bilan birga HTTP `response header`larini ham ko‘rsatadi.

```
HTTP/1.0 200 OK
Content-Type: text/plain

(Javob matni)
```

200 (200 OK) HTTP ilova layeridagi status codedir. “TCP darajasida ulanish bo‘ldi”, “HTTP response header qaytdi”, “response body kutilgan mazmunda” — bular turli layerlarga tegishli uchta alohida tekshiruvdir.

11.5 Open port va closed portni solishtirish

```
curl --max-time 2 http://127.0.0.1:18081/
sudo ss -lnt | grep ':18081'
```

Agar 127.0.0.1:18081da kutayotgan socket bo‘lmasa, ulanish odatda rad etiladi va curl Connection refused deb ko‘rsatadi. ss -lntda shu manzil va port ko‘rinmasligi u yerda TCP kutish socketi yo‘qligini bildiradi. --max-time 2 curlning butun amaliga ikki soniya chegarasi qo‘yadi.

Kutish socketi mavjudligi HTTP response ham keladi degani emas. Kutish ulanishni qabul qilishga tayyorlikdir; HTTP response uchun server programi requestni qayta ishlashi kerak. Timeout esa aloqa belgilangan vaqt ichida tugamaganini bildiradi. Bu natijaning o‘zi “kutish socketi yo‘q” degan isbot emas.

11.6 Service userning o‘qish permissioni

Veb-server processi kontent fileini clientga qaytarishi uchun, uni ishlatayotgan userda shu fileni o‘qish va yuqori directorylardan o‘tish permissionlari bo‘lishi kerak.

```
id jduweb
stat -c '%U:%G %a %n' /srv/jdu-web/index.txt
namei -l /srv/jdu-web/index.txt
sudo -u jduweb -- test -r /srv/jdu-web/index.txt
echo $?
```

jduweb nomidan su - bilan interaktiv login qila olmaslik muammo emas. service user uchun interaktiv shell ko‘pincha xavfsizlik sababli o‘chiriladi. 6-bobdagi sudo -u bilan interaktiv login ochmasdan, aynan shu user huquqida fileni o‘qib bo‘lishini sinash mumkin.

11.7 Loglar qayerdan keladi?

program ishlayotganda turli log messagelar chiqarishi mumkin. systemd-journald har xil service processlarining stdout, stderr va tizim log xabarlarini yig‘ib, ikkilik formatda saqlaydi. journalctl shu journal loglarni ko‘rish va qidirish uchun ishlatiladi.

```
curl -i http://127.0.0.1:8081/
sudo journalctl -u jdu-web.service --no-pager -n 20
```

-u natijani belgilangan unit bilan cheklaydi, -n 20 oxirgi 20 yozuvni ko‘rsatadi. --no-pager qo‘shimcha ko‘rish oynasini ochmaydi. Veb-server murojaatlarni yozadigan qilib tuzilgan bo‘lsa, requestga mos log satri qoladi. Uning mazmuni va umuman mavjudligi programning tuzilishi va sozlamasiga bog‘liq.

11.8 Web service requestga qanday javob beradi?

Veb-service misolida sozlama, bajarish, aloqa, javob va log quyidagicha bog‘lanadi.

```
Unit file → systemd → service processi
                ↓ kernel socketidan foydalanadi
          Listening socket (IP address va port)
                ↓ client ulanadi va request yuboradi
          Service processi requestni qayta ishlaydi
            ├── HTTP response qaytaradi
            └── kerak bo‘lsa log yozadi
```

unit file ishga tushish tartibini belgilaydi, lekin aloqa mazmunini service processi qayta ishlaydi. Kutish socketi ulanish nuqtasidir. HTTP response program requestni qayta ishlaganining natijasi. log esa amal yoki xatoni keyin bilish uchun yozuvdir. systemctl service holatini, ss socketni, curl HTTP javobini, journalctl yig‘ilgan logni ko‘rsatadi. Ular bitta hodisaning turli nomlari emas.

Bob yakunidagi savollar

1-savol

Service active bo‘lsa ham HTTP orqali kutilgan javob kelmasligiga ikki sabab keltiring.

2-savol

127.0.0.1:8081 va 0.0.0.0:8081 kutish manzillari tashqi tarmoqdan ulanish nuqtayi nazaridan qanday farqlanadi?

3-savol

Nima uchun ss chiqishidagi process PIDi systemd Main PIDi bilan solishtiriladi?

Bob yakunidagi javoblar

1-savol

Javob: Masalan, server programi boshqa manzil yoki portda kutayotgan bo‘lishi mumkin. Yoki service user kontent fileini o‘qiy olmagani sababli requestni to‘g‘ri bajara olmaydi.

2-savol

Javob: 127.0.0.1:8081 faqat shu host ichidagi ulanishlarni qabul qiladi. 0.0.0.0:8081 shu hostning barcha mahalliy IPv4 manzillarida kutadi. Lekin tashqi tarmoqdan haqiqiy yetib kelish firewall va tarmoq tuzilishiga ham bog‘liq.

3-savol

Javob: Kutayotgan socketdan foydalanuvchi process aynan systemd kerakli service sifatida boshqarayotgan process ekanini tekshirish uchun. Bir nechta processli serviceda kutayotgan PID Main PIDdan farq qilishi ham mumkin.

Foydalanilgan manbalar

- Linux man-pages, socket(7) (2026-09-18 kuni tekshirilgan)
- Linux man-pages, connect(2) (2026-09-24 kuni tekshirilgan)
- curl, How to use curl (2026-09-24 kuni tekshirilgan)
- Ubuntu 24.04 amaliy muhitidagi man ss, man curl, man journalctl (nashrdan oldin tekshiriladi)
- Ochiq Lab v1.0.0dagi P5/M5 program, unit va checker (Labga xos qiymatlar uchun asosiy manba)

12-bob. SSH orqali masofadan boshqarish va file uzatish

12.1 Ulanish manbasi va manzili

Masofadagi kompyuterni boshqarayotganda commandni kiritayotgan tomon bilan uni bajaradigan tomonni ajrating. SSH ulanishini boshlaydigan muhit local, ulaniladigan muhit remote deyiladi. Bu nomlar amalni bajarayotgan odamning nuqtayi nazariga bog‘liq. local doim stol ustingizdagi fizik PC emas. Agar brauzerda ochilgan bulut terminalidan SSH bajarilsa, o‘sha terminal ishlayotgan muhit ulanish manbasi bo‘ladi.

```
Command kiritiladigan muhit (local)
        ↓ SSH ulanishi
Command bajariladigan server (remote)
```

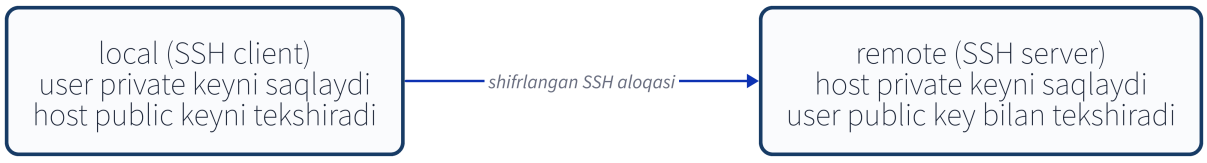
Ulanishdan oldin va keyin user nomi, hostname va ish directorysi o‘zgarishi mumkin. Masalan, avval id -un, hostname, pwd bajarilsa local holati ko‘rinadi. SSH orqali login qilingach, ayni commandlar remote holatini ko‘rsatadi. exit remote shellni tugatib, oldingi local shellga qaytaradi. Faqat prompt ko‘rinishiga qarab qaysi muhitda ekaningizni taxmin qilmang.

12.2 SSH nimani himoya qiladi?

SSH (Secure Shell) uzoqdagi kompyuter bilan shifrlangan aloqa yo‘lini yaratib, remote shell va commandlardan foydalanishga imkon beradi. Ulanishda asosan ikkita tekshiruv bor.

1. Ulaniladigan hostni tekshirish (host authentication): u haqiqatan mo‘ljallangan server ekanini host key orqali aniqlash.
2. userni tekshirish (user authentication): ulanuvchining login huquqi borligini aniqlash. public key authenticationda userning kalit jufti ishlatiladi.

Shundan keyin commandlar, natijalar va uzatiladigan ma’lumotlar shifrlangan SSH aloqasidan o‘tadi. Lekin shifrlashning o‘zi serverga yetib borishni ta’minlamaydi; ular o‘rtasida tarmoq yo‘li ham bo‘lishi kerak. SSH login muvaffaqiyatli bo‘lsa ham, remotedagi har bir fileni o‘qish yoki o‘zgartirish mumkin degani emas. Login keyingi amallarga remote OSdagi user va permissionlar qo‘llanadi.



12-1-rasm. SSHda serverni aniqlaydigan host key va userni tasdiqlaydigan user key turli vazifani bajaradi.

12.3 User key va host key qayerda saqlanadi?

public key authenticationda ulanuvchi userning private keyi SSH client tomonida saqlanadi. Unga mos public key odatda remote userning ~/.ssh/authorized_keys fileida ro‘yxatdan o‘tkaziladi. SSH client private keyni serverga bermasdan unga egaligini isbotlaydi.

host key esa serverni aniqlaydi. server host private keyni o‘zida saqlaydi. client server ko‘rsatgan host public keyni tekshiradi; ishonilgan yozuv odatda ~/.ssh/known_hostsda saqlanadi. Birinchi ulanishda ko‘rsatilgan kalit barmoq izini ishonchli boshqa yo‘ldan olingan ma’lumot bilan solishtirgach qabul qiling. Noma’lum kalitni ko‘r-ko‘rona qabul qilsangiz, soxta serverni sezmay qolishingiz mumkin.

Kalit turi	Asosiy maqsad	private key joyi	public keyga oid ma’lumot
user key	Ulanuvchi userning authenticationi	local	remotedagi authorized_keys
host key	Ulaniladigan serverni tekshirish	remote	localdagi known_hosts va boshqa yozuvlar

private keyni e’lon qilmang va Git repositoryga qo‘shmang. Boshqa userlar o‘qiy oladigan private key fileini SSH client ishlatishni rad etishi mumkin. Jadvaldagi ~ har bir muhitdagi userning home directorysini bildiradi. Belgisi bir xil bo‘lsa ham, local va remoteda boshqa joyni ko‘rsatadi.

12.4 Ulanish nomi va tarmoq yo‘li

SSH client har bir ulanish uchun sozlamani localdagi ~/.ssh/configga yozishi mumkin. Quyidagi namuna shunday sozlamadir.

```
Host study-server
  HostName server.example.edu
  User learner
  IdentityFile ~/.ssh/id_ed25519
```

Shu sozlama bilan ssh study-server yozilsa, SSH client study-serverga bog‘langan hostname, user va private key fileini ishlatadi. Host qiymati SSH sozlamasidagi host alias bo‘lib, DNSda ro‘yxatga olingan hostnamedan farq qiladi. Shu sababli SSHda ishlaydigan alias SSH sozlamasini o‘qimaydigan boshqa programda nomga aylantirilmasligi mumkin.

remote tarmoqqa bevosita yetib bo‘lmasa, SSH aloqasi oraliq host yoki uzatuvchi program orqali o‘tishi mumkin. OpenSSHda yo‘lni ProxyJump yoki ProxyCommand bilan belgilash mumkin. Bu “ulanish qanday yetib boradi?” savoliga javob; “kim login qila oladi?” degan user authentiondan boshqa masala. To‘g‘ri private key bo‘lsa ham tarmoq yo‘li bo‘lmasa, ulanish amalga oshmaydi.

12.5 Interaktiv login va remote command

ssh study-server muvaffaqiyatli authentiondan keyin remotedagi interaktiv shellni ochadi. SSH commandidan keyin bajariladigan matn yozilsa, u faqat remoteda bajariladi va natija localga qaytariladi.

```
ssh study-server 'id -un; hostname; pwd'
```

Bu misoldagi id -un, hostname, pwd remoteda ishlaydi. Tashqi yakka qo‘shtirnoqlar local shellning $HOME yoki $(...)ni muddatidan oldin kengaytirishiga ham yo‘l qo‘ymaydi. Masalan, ssh study-server 'printf "%s\n" "$(hostname)" "$HOME"'da remote shell hostname va home directoryni aniqlaydi. Tashqarisiga qo‘sh qo‘shtirnoq ishlatilsa, buni avval local shell kengaytiradi.

~ ham qaysi host va qaysi user shelli talqin qilishiga qarab boshqa home directoryni anglatadi. localdagi ~/notes.txt bilan remotedagi ~/notes.txt nomi o‘xshash, lekin boshqa filelardir.

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

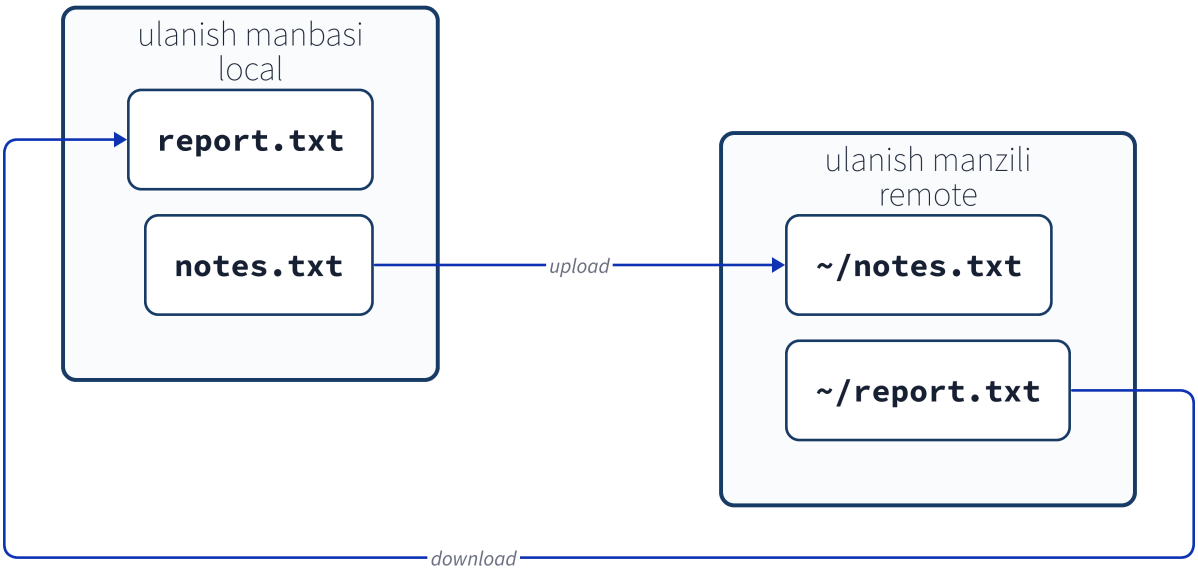
Birinchi qatorda $HOME kengaytirilishi ham, > orqali saqlash ham remoteda bajariladi; file remote userning home directorysida yaratiladi. Ikkinchi qatorda tashqi qo‘shtirnoqlardan keyingi >ni local shell talqin qiladi. remotedan qaytgan natija localdagi local-result.txtga yoziladi. Qaysi shell matnni talqin qilishi ayni belgining ta’sirini o‘zgartiradi.

12.6 SCP orqali file yuborish va qabul qilish

scp SSH aloqa va authenticationidan foydalanib file uzatadi. Umumiy ko‘rinishi scp SOURCE DESTINATION; remote path hostname:path shaklida yoziladi.

```
scp notes.txt study-server:~/notes.txt
scp study-server:~/report.txt report.txt
```

Birinchi qator localdagi notes.txtni remotega yuboradi — upload. Ikkinchisi remotedagi report.txtni localga oladi — download. :dan keyingi ~ remote userning home directorysini anglatadi. study-server kabi SSH alias scpda ham ishlaydi.



12-2-rasm. Upload va download command bajarilayotgan tomon nuqtayi nazaridan farqlanadi.

SOURCE va DESTINATIONni almashtirib yuborsangiz, file kutilganiga teskari yo‘nalishda ko‘chadi. Manzilda shu nomli file bo‘lsa ustidan yozilishi mumkin. local va remote filelari alohida file systemlarda saqlanadi.

12.7 Mazmun mosligi va file atributlari

Uzatgandan keyin ikki file mazmuni bir xil ekanini tekshirish uchun ikkalasida sha256sum bajarib, hash qiymatlarini solishtirish mumkin.

```
sha256sum notes.txt
ssh study-server 'sha256sum ~/notes.txt'
```

hash qiymatlari mos kelishi ikkala filening mazmuni bir xil ekaniga kuchli dalil. Lekin owner, group, permission mode va saqlash manzili hashga kirmaydi. Uzatilgan filening egasi yoki permissioni manzilda boshqacha bo‘lishi mumkin. Atributlarni remoteda stat bilan alohida tekshiring.

```
ssh study-server 'stat -c "%U:%G %a %n" ~/notes.txt'
```

Mazmunni tekshirish bilan atributni tekshirish turli savollarga javob beradi. sha256sum: “ichidagi ma’lumot bir xilmi?” stat: “remoteda file kimga tegishli va qanday permissionga ega?”

Bob yakunidagi savollar

1-savol

Brauzerda ochilgan bulut terminalidan boshqa serverga SSH ulansa, local va remote qaysi muhitlar bo‘ladi?

2-savol

SSH user key va host key nimani tekshiradi? User private key qayerda bo‘ladi?

3-savol

SSH host alias bilan DNS hostname farqi nima? To‘g‘ri kalit bo‘lsa ham ulanish nima uchun amalga oshmasligi mumkin?

4-savol

Quyidagi ikki commandda $HOME qayerda kengaytiriladi va natija qayerda saqlanadi?

```
ssh study-server 'printf "%s\n" "$HOME" > "$HOME/home-path.txt"'
ssh study-server 'printf "%s\n" "$HOME"' > local-result.txt
```

5-savol

scp bilan uzatilgan ikki filening SHA-256 qiymati bir xil. Egasi va permission mode ham bir xil deyish mumkinmi?

Bob yakunidagi javoblar

1-savol

Javob: SSH commandi ishlayotgan bulut terminal muhiti local, ulaniladigan server esa remote. Brauzerni ko‘rsatayotgan fizik PC har doim local bo‘lavermaydi.

2-savol

Javob: user key ulanuvchi userni tasdiqlaydi; host key ulaniladigan serverni tekshiradi. user private key localda saqlanadi va remotega berilmaydi.

3-savol

Javob: host alias localdagi SSH sozlamasida yozilgan boshqa nom, u DNS hostname bo‘lishi shart emas. Kalit to‘g‘ri bo‘lsa ham, remotegacha tarmoq yo‘li bo‘lmasa SSH ulana olmaydi.

4-savol

Javob: Ikkalasida ham $HOME remoteda kengaytiriladi. Birinchi qatordagi > qo‘shtirnoq ichida bo‘lgani uchun remoteda yozadi. Ikkinchisidagi > qo‘shtirnoq tashqarisida bo‘lgani uchun localda yozadi.

5-savol

Javob: Yo‘q. hash file mazmunini solishtiradi, egasi va permission modeni o‘z ichiga olmaydi. Ularni manzilda stat bilan alohida tekshiring.

Foydalanilgan manbalar

- Ubuntu Server documentation, OpenSSH server (2026-09-24 kuni tekshirilgan)
- OpenBSD manual pages, ssh(1), ssh_config(5), scp(1) (2026-09-24 kuni tekshirilgan)
- GNU Coreutils, sha256sum(1) (2026-09-24 kuni tekshirilgan)

Kitob yakuni

Kompyuterdagi programlar OS taqdim etadigan imkoniyatlar yordamida ishlaydi. Linuxda ishlaganda faqat command nomini emas, bajarilayotgan amal file, user, process yoki aloqa bilan qanday bog‘liqligini o‘ylang. Shunda natijani tushunish osonlashadi.

Sozlama fileining mavjudligi, service processining ishlab turishi va tarmoqdan kutilgan javobni olish — turli holatlar. Ulardan bittasi to‘g‘ri bo‘lsa, qolganlari ham albatta to‘g‘ri degani emas. file egasi va permissioni, servicening ishlayotgan useri, kutish socketi va logni bir-biridan ajratib o‘qing.

Keyingi darslarda Nginx, BIND 9 va PostgreSQL kabi turli vazifadagi dasturlar ko‘riladi. Ularning sozlanishi boshqa-boshqa. Ammo “qaysi user nomidan ishlaydi?”, “qaysi filelardan foydalanadi?”, “aloqani qaysi manzilda qabul qiladi?”, “holatini qayerdan tekshirsa bo‘ladi?” degan savollar har biriga mos keladi. Ushbu kitobdagi OS tushunchalari yangi tizimni anglash uchun asos bo‘lsin.

Command yozilishi yoki option ma’nosini eslash kerak bo‘lsa, [atamalar va commandlar ma’lumotnomasiga](../reference.md) qarang. Unda commandni o‘qish, find, grep va tail bo‘limlari qisqa misollar orqali id -unni hamda qidirishning boshlanish joyi, sharti va chiqish amalini tushuntiradi.